AppComm

SRS Chat-Robot

A customer relation management application to get a better and faster overview and to manage customers more efficiently and effectively

Moritz Frieling
23.10.2023

Table of content

A. List of Tables 3

B. List of Figures 3

1. Introduction 4

1.1 Purpose 4

1.2 Scope 4

1.3 Definitions, acronyms and abbreviations 5

1.4 References 6

1.5 Overview 6

2 Overall Description 7

2.1 Product Perspective 7

2.1.1 **System Interfaces** 7

2.1.2 **User Interfaces** 7

2.1.3 **Hardware interfaces** 7

2.1.4 **Software interfaces** 8

2.1.5 **Communication interfaces** 8

2.1.6 **Memory** 8

2.1.7 **Operations** 8

2.1.8 **Site adaption requirements** 8

2.2 Product functions 8

2.2.1 **User groups and stories** 9

2.2.2 **Category 1: CRUD Customer + (De)activation** 13

2.2.3 **Category 2: CRUD Solution + (Un)assign + Overview** 19

2.2.4 **Category 3: CRUD Contact Person** 26

2.2.5 **Category 4: Use Case(s) Ticket System** 29

2.3 User characteristics 32

2.4 Constraints 32

2.4.1 **Regulatory policies** 32

2.4.2 **Interfaces to other applications** 32

2.4.3 **Parallel operation** 32

2.4.4 **Reliability requirements** 32

2.4.5 **Criticality of the application** 33

2.4.6 **Safety and security considerations** 33

2.5 Assumptions and dependencies 33

2.6 High-order Language Requirements 33

2.7 Apportioning of requirements 33

3 Specific requirements 34
3.1 External interfaces 34
3.1.1 **User Interfaces** 34
3.2 Functions 43
3.2.1 Requirements and their relationship towards the Use Cases 43
3.2.2 Enter new information 43
3.2.3 Change existing information 45
3.2.4 View information 45
3.2.5 Deleting information 46
3.3 Performance requirements 48
3.3.1 **Search feature** 48
3.3.2 **Response time: User input** 48
3.3.3 **Fast updates** 48
3.3.4 **System stability: Simultaneous users** 48
3.3.5 **Response time: Search function** 49
3.3.6 **System stability: Simultaneous entries** 49
3.3.7 **User Experience** 49
3.4 Logical database requirements 49
3.5 Design constraints 50
3.5.1 **Memory Usage** 50
3.6 Software system attributes 50
3.6.1 **Reliability** 50
3.6.2 **Availability** 51
3.6.3 **Security** 51
3.6.4 **Maintainability** 51
3.6.5 **Portability** 51
3.7 Organising the specific requirements 52
3.8 Additional comments 52
3.9 Version History 52
Appendixes 53
Test Scenarios 53

A. List of Tables

Table 1 Definitions 5
Table 2 References 6
Table 3 User Stories and Their Use Cases 12
Table 4 Relationship between the Functional requirements and the use cases 43
Table 5 Version History 52

B. List of Figures

Figure 1 Use Case Diagram 11
Figure 2 Customer Overview 34
Figure 3 Create Customer 35
Figure 4 Read Customer 35
Figure 5 Update Customer 36
Figure 6 Delete Customer 36
Figure 7 (De)Activate customer 36
Figure 8 Create Contact Person 37
Figure 9 Read Contact Person 37
Figure 10 Update Contact Person 38
Figure 11 Confirm Update Contact Person 38
Figure 12 Confirm Delete Contact Person 39
Figure 13 Solution Overview 39
Figure 14 Create Solution 40
Figure 15 Read Solution 40
Figure 16 Update Solution 41
Figure 17 Delete Solution 42
Figure 18 Create Ticket 42
Figure 19 Domain Model 50

1. Introduction

In the following subsections, you will get an overview of the entire SRS.

1.1 Purpose

The SRS you are about to read was created with the intend of explaining why the project is necessary, what the project exactly is, what will and won't be in the scope of the project and any other information relevant for any stakeholders. Furthermore, this document should be readable for the customers but also for the developers since it is written in a way easy to understand for any non-developer but gives the developers enough information to be able to get a quick overview over the project.

This means, that the intended audience of the SRS are the developers, testers, project managers and customers of the project. With customers, for this project AppComm itself is meant.

1.2 Scope

AppComm has a lot of different customers, with vastly different services, products and needs. Since there are a lot of different data streams, AppComm has issues with managing all their customers and because of that, it's hard to help customers quickly, get an overview of all their customers, and manage them effectively.

The project "ChatRobot" will make it easier to manage customers data. Due to the different data streams and sources, the projects main aim is to combine the different sources into one CRM (Customer Relationship Management) tool, specifically all their general information, needed to contact and communicate, but also all the developed products and services that the customer uses.

The product that will be delivered consists of a web application in PHP Laravel. To set up this website, a database is used, which will be decoupled later and swapped out with an internal „Robot" of AppComm. Accessing this robot and interacting with it is not part of this project. Nor is a tested rollout or any kind of beta test part of the scope.

The 5 developers from Fontys will create a CRM tool for AppComm within the four months from September to December. The product that is to be delivered will help manage the customers of AppComm and give a better overview due to displaying all customers and their information at one place. Based on the requirements set by AppComm, the bi-weekly sprints

need to be carried out by applying Test-Driven Development, using the required frameworks and languages, and satisfying the customers need, which have been found out about mainly in the analysis phase of the project. Further information can be found in the SRS, where the specific requirements are mentioned, which can be used as performance specifications for the project.

1.3 Definitions, acronyms and abbreviations

Abbreviation, Acronym	Definition
SRS	**S**oftware **R**equirements **S**pecification
CR	**C**hat **R**obot
Entry	An Entry can be a customer, a solution or even a contact person, so every entity within the system
User	A user can be either a customer or an administrator of the application
GDPR	**G**eneral **D**ata **P**rotection **R**egulation
API	**A**pplication **P**rogramming **I**nterface
UC	Short term for “use case”
Fig.	Short term for “figure”
DESC	Short term for “description”
RAT	Short term for “rational”
DEP	Short term for “dependency”
FR	Short term for “functional requirement”
NFR	Short term for “non-functional requirement”
NFR	Short term for “non-functional requirement”

TABLE 1 DEFINITIONS

1.4 References

Title	Publisher	Source	Publishing Date
IEEE template	IEEE	https://www.ieee.org/conferences/publishing/templates.html	25 June 1998
GDPR	EU	https://gdpr.eu/data-privacy/	25. May 2018

TABLE 2 REFERENCES

1.5 Overview

Chapter 2 and 3 of the SRS contain the requirements of the project. For this, chapter 2 provides the reader with some general information about the product. Subchapter 2.2 will then describe the business requirement of the TCT. The specific functional and non-functional requirements of the TCT can be found in chapter 3.

2 Overall Description

This section of the SRS should describe the general factors that affect the product and its requirements. It does not state specific requirements. Instead, it provides a background for those requirements, which are defined in detail in Section 3 of the SRS and makes them easier to understand.

2.1 Product Perspective

Currently, AppComm employees face issues with getting information about their clients, including adding, changing or deleting those information. There are a lot of different data sources and managing the customers information effectively and helping/ answering the customers quickly is one of the problems that AppComm has due to that.

By implementing the CRM, AppComm hopes to improve their effectiveness and speed when it comes to the management of those information.

2.1.1 System Interfaces

Since the first version of the “ChatRobot” project will work independent and does not rely on a larger system, this subchapter is not applicable in this SRS at this point in time. Further versions of the project might require external interfaces, so this section might have to be updated in further iterations.

2.1.2 User Interfaces

The GUI shall be easy to read and navigate. The start view will be a welcome page for the individual. Since it shall be as easy as possible to enter or view information and as hard as possible to make mistakes, there should be a list with options to choose from, whenever possible. There will still be manual input, but it should be reduced.

An administrator, a person who can create, alter or delete content from all customers within the application, shall be able to do so within 10 minutes of getting to know the application.

A customer, a person who wants to view and possibly use the information within the system, shall be able to do so within 5 minutes of getting to know the application.

2.1.3 Hardware interfaces

The application will require a screen, keyboard and a mouse or a touchscreen on mobile devices to navigate and to receive in- and output.

2.1.4 Software interfaces

When the user's hardware fulfils all hardware interface requirements (2.1.3), the device also needs a stable internet connection as well as a web-browser.

2.1.5 Communication interfaces

The exchange of information between the parts of the whole system is crucial since they inevitably depend on each other. However, in which way that communication is achieved, at example which network protocols to use, is not of importance for the system and is therefore handled by the underlying operating systems.

2.1.6 Memory

The customer had no requirements regarding the memory, but when developing the TCT, the developers could ensure that both primary and secondary memory won't be used too much.

2.1.7 Operations

Since the customer data doesn't lie directly within the application but gets pulled from the database and in the future possibly the "robot", both data sources could be accessed directly if that is necessary. A backup system is not part of the project, but rollback shall be possible.

2.1.8 Site adaption requirements

Since the system is based on a lot of data from the customers, supplied by the administrator or the customers themselves, this data needs to be stored. Even though after the successful project, the data source will be changed, for the time being an additional database is necessary to test the functionalities of the system. The web application will provide data to it and get information back. Both the application as well as the database will be part of this iteration, even though this database doesn't serve a use for the customer other than testing.

2.2 Product functions

To gather the functions the product has to deliver, user groups as well as user stories had to be created, which can be seen in the following section.

2.2.1 User groups and stories

2.2.1.1 User Groups

1. Administrator: Any employee of AppComm who has access to the software is an Administrator, being able to create, view, change and delete the customer's data and communicate with them.
2. A customer is a customer from AppComm with a customer-access to this software, enabling him to see an overview and communicate with the administrators.

The customer has an overview about himself and can change some of his information within the application, the specific actions being defined below. The administrator can view all customers and their information and has also additional possible actions.

At this point in time, the User Groups are not yet implemented, since only the tasks of the administrator have been tackled. The implementation of the User Groups will take place in the next iteration of the project.

2.2.1.2 User Stories

In the following, all User Stories can be found. It is relevant to know that since there are some User Stories where both the Administrator as well as the customer have the exact same flow of events, those User Stories can be found combined in the end.

1. As an Administrator I want to be able to...

1.1 Customer

1.1.1. see all the customers to get a good overview.

1.1.2. add a customer if there is a new customer for AppComm.

1.1.3. view, edit or delete an existing customer to manage them, possibly if there has been a change.

1.1.4. (De)activate a customer to enable/ disable access.

1.1.5. [add a quick note for a specific customer to quickly write something down.]

1.2 Contact Person

1.2.1. add a contact person related to a specific customer if there is a new responsible person.

1.2.2. view, edit or delete a contact person related to a specific customer to manage them, possibly if there has been a change.

1.3 Solution

1.3.1. see all the solutions to have a good overview of them.

1.3.2. add a solution (services, products, hosting plans and domain names) and its price for that to the system if a new solution exists.

1.3.3. view, edit or delete a solution within the system to manage existing solutions.

1.3.4. assign or unassign a solution to a customer to keep an overview of which customer has which solutions.

1.3.5. [get notified when a customer has a problem with one of their products or services to deal with this problem.]

1.3.6. [set the current status of a solution for customer visibility.]

1.3.7. [upload a new invoice to a customer, so they can keep track of their expenses.]

1.4 Tickets

1.4.1. Unassign a GitLab project from/to a customer so that potential tickets can be created.

1.4.2. Write a ticket for a solution so that potential features can be implemented and issues can be solved.

1.4.3. [see all the tickets from all customers to see if there are any problems that need to be solved.]

1.4.4. [see previous invoices of a customer to have a better overview of their expenses.]

2. As a customer I want to be able to...

2.1 Solutions

2.1.1. [see all my solutions (services, products, hosting plans and domain names) and their prices in one place to have a better overview.]

2.1.2. [see the progress of my products to know how far they are.]

2.1.3. [see what kind of products were build for me and what they are for to have a better overview.]

2.1.4. [see my previous invoices to have a better overview of expenses.]

2.2 Ticket

2.2.1. [create new tickets when I have a problem with my products or services to get help more quickly.]

3. As a user (Admin and customer) I want to be able to …

3.1 Ticket

3.1.1. [write comments on a ticket to communicate with the other part.]

3.1.2. [see the comments and status on a ticket to know the progress.]

3.1.3. [set a ticket to solved when the problem is solved.]

3.2 Chat

3.2.1. [chat with the other party to have a more informal way of communicating.]

3.3 Account

3.3.1. [log in and out to be able to use the system the way it is intended.]

As can be seen in the user stories, the key functionality of the product for the administrator is viewing individual customers, their specific information and solutions as well as a cumulated view of all customers through the use of a general overview page. To do so, those customers and solutions need to be created at first. Since customers, their solutions or other information might need to be added or an entry simply is outdated, an administrator shall be able to alter or delete the specific entries. Even though the customer can change certain information, his main functionality is the overview about the projects done by AppComm as well as the communication to them.

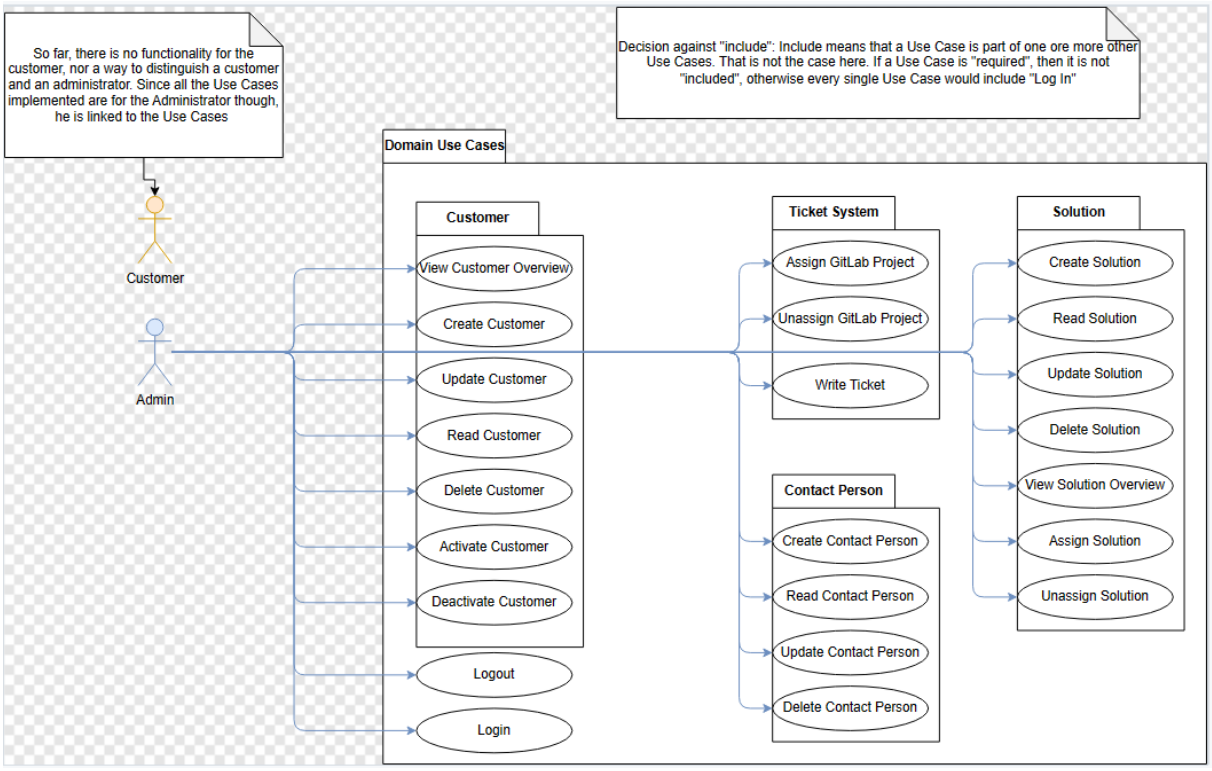


FIGURE 1 USE CASE DIAGRAM

As mentioned, there are two User Groups, but the customer has not yet been implemented. The Use Case Diagram displays all implemented Use Cases. Most of them are part of the overall “Domain Use Cases”, such as “Ticket System”, “Solution”, “Contact Person” and “Customer”, while there are also the Use Cases “Login” and “Logout”.

In the table below, these use cases can be seen with the user story they are tasked to solve and one can see whether they extend or include another use case.

User Story	Use Case
“Add Customers”	UC1.1 – Create Customer
“View, Edit and Delete Customer”	UC1.2 – Read Customer
“View, Edit and Delete Customer”	UC1.3 – Update Customer
“View, Edit and Delete Customer”	UC1.4 – Delete Customer
“(De)activate Customers”	UC1.5 – Deactivate Customer
“(De)activate Customers”	UC1.6 - Activate Customer
“View Customers”	UC1.7 - View Customer Overview
“Add Solution”	UC2.1 – Create Solution
“View, Edit and Delete Solution”	UC2.2 – Read Solution
“View, Edit and Delete Solution”	UC2.3 – Update Solution
“View, Edit and Delete Solution”	UC2.4 – Delete Solution
“View Solutions”	UC2.5 – View Solution Overview
“(Un)assign Solution”	UC2.6 – Assign Solution
“(Un)assign Solution”	UC2.7 Unassign Solution
“Add Contact Person”	UC3.1 – Create Contact Person
“View, Edit and Delete Contact”	UC3.2 – Read Contact Person
“View, Edit and Delete Contact”	UC3.3 – Update Contact Person
“View, Edit and Delete Contact”	UC3.4 – Delete Contact Person
“(Un)assign GitLab Project”	UC4.1 Assign GitLab Project
“(Un)assign GitLab Project”	UC4.2 Unassign GitLab Project
“Write Ticket”	UC4.3 Write Ticket

TABLE 3 USER STORIES AND THEIR USE CASES

For all use cases which are listed below, it is possible to go back to the home screen or close the window. Any progress, at example when adding or changing information, will be lost, if this progress was not saved successfully before. This Table does not contain any “Extends” or “Includes” since until now, no Use Case needed that.

In the chapter “Appendixes” under “Testing Section”, the test scenarios for all of the use cases can be found.

2.2.2 Category 1: CRUD Customer + (De)activation

Use Case 1.1: Create Customer

Name:	***Create Customer***
Actor:	Administrator
Description:	Create a customer profile with general information
Pre-condition:	Administrator is on general Customer Overview page.
Scenario:	1. Administrator indicates that a new customer needs to be added. 2. System displays general information form, containing: Company (Name), Registration Number(unique), VAT Number, adress (Country, City, ZIP Code, Street, number, additional adress information(optional)), general mail adress, phone number (optional), website(optional) 3. Administrator fills the necessary fields in and saves the form. 4. System indicates that the customer has been saved.
Results:	A new customer has successfully been added.
Exceptions:	4.a. Missing data 4.a.1 System indicates that data is missing in certain fields. 4.a.2 Go back to step 2. 4.b. Incorrect data 4.b.1 System indicates data is incorrect in certain fields. 4.b.2 Go back to step 2.

Name:	***Create Customer***
	4.c. Customer already exists
	4.c.1 System indicates that the registration number is already present in system.
	4.c.2 Go back to step 2.

Use Case 2.1 Read Customer

Name:	***Read Customer***
Actor:	Administrator
Description:	Administrator looks at specific Customer
Pre-condition:	Administrator is on the general overview page of all customers
Scenario:	1. System displays the general overview page.
	2. Administrator clicks on specific customer
	3. System opens this specific customers page and displays the information to the administrator.
Results:	Administrator can see a customers information

Update Customer

Name:	***Update Customer***
Actor:	Administrator
Description:	Administrator wants to change a specific customer's information
Pre-condition:	Administrator is on the page of a specific customer

Name:	***Update Customer***
Scenario:	1. Administrator indicates that the customer information need to be updated
	2. System makes the fields editable
	3. Administrator makes changes to the fields and indicates the system to save the changes.
	4. System indicates that the customer has been saved.
Results:	The customer's information got altered successfully.
Exceptions:	4.a. Missing data
	4.a.1 System indicates that data is missing in certain fields.
	4.a.2 Go back to step 2.
	4.b. Customer already exists
	4.b.1 System indicates that the Registration Number of the customer is already present in system.
	4.b.2 Go back to step 2.
	4.c. Incorrect data
	4.c.1 System indicates dats is incorrect in certain fields.
	4.c.2 Go back to step 2.

Delete Customer

Name:	***Delete Customer***
Actor:	Administrator

Name:	***Delete Customer***
Description:	Administrator wants to delete customer from the system
Pre-condition:	Administrator is on the update page of a specific customer
Scenario:	1. Administrator indicates that specific customer should be deleted.
	2. System warns the administrator by pointing out that this can't be undone.
	3. Administrator confirms the deletion.
	4. System indicates that the customer has been deleted, with all the information present.
Results:	Customer was successfully deleted.
Exceptions:	3.a. Administrator cancels the process
	3.a.1 Administrator denies the deletion.
	3.a.2 Use Case ends here.

Deactivate Customer

Name:	***Deactivate Customer***
Actor:	Administrator
Description:	Customer should lose access to the system
Pre-condition:	Administrator is on the update page of a specific customer
Scenario:	1. Administrator indicates customer should be deactivated.

Name:	***Deactivate Customer***
	2. Sytem warns the administrator and points out this deactivation can be undone.
	3. Administrator confirms the deactivation.
	4. System indicates that the customer was deactivated.
Results:	Customer was succesfully deactivated.
Exceptions:	3.a. Admin cancells process
	3.a.1 Administrator denies the deactivation.
	3.a.2 Use Case ends here.

Activate Customer

Name:	***Activate Customer***
Actor:	Administrator
Description:	Customer should regain access to the system
Pre-condition:	Administrator is on the update page of a specific customer
Scenario:	1. Administrator indicates customer should be activated.
	2. Sytem asks the admin to make sure this action is correct.
	3. Administrator confirms the activation.
	4. System indicates that the customer was activated.
Results:	Customer was succesfully activated.
Exceptions:	3.a. Admin cancells process

Name:	***Activate Customer***
	3.a.1 Administrator denies the activation.
	3.a.2 Use Case ends here.

View Customers Overview

Name:	***View Customers Overview***
Actor:	Administrator
Description:	View, filter and sort multiple customers
Pre-condition:	Administrator is logged in.
Scenario:	1. Administrator indicates he wants to go to the general Customer Overview Page.
	2. System displays all the customers with their following information: company title, registration number, location details (in one field)
	3. Admin views all the information
Results:	
Extension:	3.a. Administrator wants to filter
	3.a.1 Admin indicates he wants to filter the customers.
	3.a.2 System gives the admin the option to filter the customers based on the fields displayed.
	3.a.3 Admin selects the specific filters.
	3.a.4 Return to step 1.
	3.b. Administrator wants to sort

Name:	***View Customers Overview***
	3.b.1 Admin indicates he wants to sort the customers.
	3.b.2 System gives the admin the option to sort the customers based on the fields displayed.
	3.b.3 Admin selects the specific sorting option.
	3.b.4 Return to step 1.

2.2.3 Category 2: CRUD Solution + (Un)assign + Overview

Create Solution

Name:	***Create Solution***
Actor:	Administrator
Description:	Create a Solution with general information for a specific customer
Pre-condition:	Administrator is on the general Solution overview page
Scenario:	1. Administrator indicates that a new Solution needs to be added
	2. System displays general information form, containing: number(plain number, unique), category(service/product/domain name), title, description.
	3. Administrator fills the necessary fields in and saves the form.
	4. System indicates that the Solution has been saved.
Results:	A new Solution has successfully been added to the database.
Exceptions:	4.a. Missing data
	4.a.1 System indicates that data is missing in certain fields.

Name:	***Create Solution***
	4.a.2 Go back to step 2.
	4.b. Incorrect data
	4.b.1 System indicates dats is incorrect in certain fields.
	4.b.2 Go back to step 2.
	4.c. Solution already exists
	4.c.1 System indicates Solution number is already present in system.
	4.c.2 Go back to step 2.

Read Solution

Name:	***Read Solution***
Actor:	Administrator
Description:	Administrator looks at specific Solution
Pre-condition:	Administrator is on the general Solution overview page
Scenario:	1. System displays the overview page.
	2. Administrator clicks on specific Solution
	3. System opens this specific Solution's page and displays the information to the administrator.
Results:	Administrator can see a Solutions information

Update Solution

Name:	***Update Solution***
Actor:	Administrator
Description:	Administrator wants to change information of a specific Solution
Pre-condition:	Administrator is on a specific Solution overview page
Scenario:	1. Administrator indicates that the Solution information need to be updated
	2. System makes the fields editable
	3. Administrator makes changes to the fields and indicates the system to save the changes.
	4. System indicates that the Solution has been saved.
Results:	The Solution's information got altered successfully.
Exceptions:	4.a. Missing data
	4.a.1 System indicates that data is missing in certain fields.
	4.a.2 Go back to step 2.
	4.b. Incorrect data
	4.b.1 System indicates dats is incorrect in certain fields.
	4.b.2 Go back to step 2.
	4.c. Solution already exists
	4.c.1 System indicates that the updated Solution is already present in system.

Name:	***Update Solution***
	4.c.2 Go back to step 2.

Delete Solution

Name:	***Delete Solution***
Actor:	Administrator
Description:	Administrator wants to delete Solution from the system
Pre-condition:	Administrator is on a specific Solution update page
Scenario:	1.Administrator indicates that specific Solution should be deleted.
	2. System warns the administrator by pointing out that this can't be undone.
	3. Administrator confirms the deletion.
	4. System indicates that the Solution has been deleted, with all the information present.
Results:	Solution was successfully deleted.
Exceptions:	2.a. Customer still assigned
	2.a.1 Systems warns that the solution cannot be deleted because it's still assigned
	2.a.2 Use Case ends here.
	3.a. Administrator cancels the process
	3.a.1 Administrator denies the deletion.
	3.a.2 Use Case ends here.

View Solution Overview

Name:	***View Solution Overview***
Actor:	Administrator
Description:	View, filter and sort multiple solutions
Pre-condition:	Administrator is logged in.
Scenario:	1. Administrator indicates he wants to view the general Solution Overview page. 2. System displays all the solutions with their following information: number(plain number, unique), category(service/product/domain name), title. 3. Admin views all the information .
Results:	Administrator can sucessfully view Solution Overview
Extension:	3.a. Administrator wants to filter 3.a.1 Admin indicates he wants to filter the solutions. 3.a.2 System gives the admin the option to filter the solutions based on the fields displayed. 3.a.3 Admin selects the specific filters. 3.a.4 Return to step 1. 3.b. Administrator wants to sort 3.b.1 Admin indicates he wants to sort the solution. 3.b.2 System gives the admin the option to sort the solutions based on the fields displayed.

Name:	***View Solution Overview***
	3.b.3 Admin selects the specific sorting option.
	3.b.4 Return to step 1.

Assign Solution

Name:	***Assign Solution***
Actor:	Administrator
Description:	Administrator wants to assign a Solution to a customer
Pre-condition:	Administrator is on the overview page of a specific customer
Scenario:	1.Administrator indicates that a solution needs to be assigned to a customer.
	2. System displays a list of available solutions.
	3. Administrator selects the desired solution to assign.
	4. System displays additional price information fields (required now): price, units, recurring period, unit of recurring periods, one time payment, VAT rate, start and end date, additional information (free form text input).
	5. Administrator fills out the fields and saves it.
	6. System associates the solution with the customer
Results:	Solution was successfully assigned to a customer.
Exceptions:	6.a. Missing data
	6.a.1 System indicates that data is missing in certain fields.

Name:	***Assign Solution***
	6.a.2 Go back to step 4.
	6.b. Incorrect data
	6.b.1 System indicates dats is incorrect in certain fields.
	6.b.2 Go back to step 4.

Unassign Solution from Customer

Name:	***Unassign Solution from Customer***
Actor:	Administrator
Description:	Administrator wants to unassign Solution from a customer
Pre-condition:	Administrator is on the overview page of a specific customer
Scenario:	1. Administrator indicates that a solution should be unassigned from a customer.
	2. System asks for confirmation of the unassignment.
	3. Administrator confirms the unassignment.
	4. System disassociates the solution from the customer.
Results:	The solution has been successfully unassigned from the customer.
Exceptions:	3.a. Administrator cancels the process
	3.a.1 Administrator denies the confirmation request.
	3.a.2 Use Case ends here.

2.2.4 Category 3: CRUD Contact Person

Create Contact Person

Name:	***Create Contact Person***
Actor:	Administrator
Description:	Create a Contact person profile with general information
Pre-condition:	Administrator is on customer's specific page
Scenario:	1. Administrator indicates that a new contact person needs to be added
	2. System displays general information form, containing: Last Name, First Name, Middle name(s), email(unique), phone number, Function/ Role, description (relational)
	3. Administrator fills the necessary fields in and saves the form.
	4. System indicates that the contact person has been saved.
Results:	A new contact person has successfully been added to the database.
Exceptions:	4.a. Missing data
	4.a.1 System indicates that data is missing in certain fields.
	4.a.2 Go back to step 2.
	4.b. Incorrect data
	4.b.1 System indicates dats is incorrect in certain fields.
	4.b.2 Go back to step 2.
	4.c. Contact person already exists

Name:	***Create Contact Person***
	4.c.1 System indicates contact person is already present in system (due to the same email-adress).
	4.c.2 Go back to step 2.

Read Contact Person

Name:	***Read Contact Person***
Actor:	Administrator
Description:	Administrator looks at specific contact person
Pre-condition:	Administrator is on the page of a specific customer
Scenario:	1. System displays the customer's page.
	2. Administrator indicates he wants to view the contact person's information.
	3. System opens this specific person's page and displays the information to the administrator.
Results:	Administrator can see a contact person's information

Update Contact Person

Name:	***Update Contact Person***
Actor:	Administrator
Description:	Administrator wants to change information of a specific contact person of a customer
Pre-condition:	Administrator is on the page of a contact person of a specific customer

Name:	***Update Contact Person***
Scenario:	1. Administrator indicates that the contact person's information need to be updated
	2. System makes the fields editable
	3. Administrator makes changes to the fields and indicates the system to save the changes.
	4. System indicates that the contact person has been saved.
Results:	The contact person's information got altered successfully.
Exceptions:	4.a. Missing data
	4.a.1 System indicates that data is missing in certain fields.
	4.a.2 Go back to step 2.
	4.b. Incorrect data
	4.b.1 System indicates dats is incorrect in certain fields.
	4.b.2 Go back to step 2.
	4.c. Contact person already exists
	4.c.1 System indicates that the updated contact person is already present in system.
	4.c.2 Go back to step 2.

Delete Contact Person

Name:	***Delete Contact Person***
Actor:	Administrator

Name:	***Delete Contact Person***
Description:	Administrator wants to delete Solution from the system
Pre-condition:	Administrator is on the update page of a contact person of a specific customer
Scenario:	1.Administrator indicates that the contact person should be deleted. 2. System warns the administrator by pointing out that this can't be undone. 3. Administrator confirms the deletion. 4. System indicates that the contact has been deleted, with all the information present.
Results:	Contact person was successfully deleted.
Exceptions:	3.a. Administrator cancels the process 3.a.1 Administrator denies the deletion. 3.a.2 Use Case ends here.

2.2.5 Category 4: Use Case(s) Ticket System

Assign GitLab Project

Name:	***Assign GitLab Project***
Actor:	Administrator
Description:	Administrator wants to assign a GitLab project to a customer
Pre-condition:	Administrator is on the overview page of a specific customer

Name:	***Assign GitLab Project***
Scenario:	1. Administrator indicates that a GitLab project needs to be assigned to a customer.
	2. System displays a list(dropdown) of all available GitLab projects of AppComm.
	3. Administrator selects the desired GitLab project and indicates to assign it.
	4. System associates the GitLab project with the customer.
Results:	GitLab project was successfully assigned to a customer.
Exceptions:	3.a. Cancellation of process
	3.a.1 Administrator selects the project and then cancels.
	3.a.2 Use Case ends here.

Unassign GitLab Project

Name:	***Unassign GitLab Project***
Actor:	Administrator
Description:	Administrator wants to unassign a previously assigned GitLab project from a customer
Pre-condition:	Administrator is on specific customer's page
Scenario:	1. Administrator indicates a specifc GitLab project should be unassigned from a customer.
	2. System asks for confirmation of the unassignment.
	3. Administrator confirms the unassignment.

Name:	***Unassign GitLab Project***
	4. System disassociates the GitLab project from the customer.
Results:	The GitLab project has been successfully unassigned from the customer.
Exceptions:	3.a. Administrator cancels the process
	3.a.1 Administrator denies the confirmation request.
	3.a.2 Use Case ends here.

Write Ticket

Name:	***Write Ticket***
Actor:	Administrator
Description:	Administrator wants to add a new ticket via GitLab
Pre-condition:	Administrator is on specific customer's page
Scenario:	1. Administrator indicates a new ticket should be created in GitLab for the specific customer.
	2. Sytem displays a list(dropdown) of assigned GitLab projects for this customer as well as a form with a title, a category and a description for the ticket.
	3. Administrator chooses the specific GitLab project, fills in the fields, chooses the category and saves the ticket.
	4. System indicates that the ticket was created in GitLab.
Results:	Ticket was succesfully created.
Exceptions:	4.a. Missing data
	4.a.1 System indicates that data is missing in certain fields.

Name: ***Write Ticket***

4.a.2 Go back to step 2.

2.3 User characteristics

Adding, altering and deleting the information will mostly be done by the administrators. Looking up information will be done by both the customer as well as the administrator. Since all of the administrators are employees of AppComm, they have a general technical understanding, have a medium to high educational level and are experienced in their field. The customers of AppComm have different backgrounds, so a high technical expertise can't be guaranteed. Since the speed of creating, changing, deleting and looking up information is of importance, the application shall be fast and easy to use even though the employees would be able to deal with a more complex program, but that would simply take more time. That also means that customers should find everything intuitively, without having to ask for help from AppComm.

2.4 Constraints

This subsection of the SRS should provide a general description of any other items that will limit the developer's options.

2.4.1 Regulatory policies

The customer needs to fulfil the GDPR after the software is delivered successfully. Since the database will contain sensitive information about both AppComm as well as their customers, any breaches need to be handled properly and according to the GDPR norms, since the data is not anonymous.

2.4.2 Interfaces to other applications

Interfaces to other applications, like to a "QR Code Scanner", might be added in future versions of the website, but are not in the scope of this SRS.

2.4.3 Parallel operation

Even though the traffic on this application won't be high in the beginning, it might be the case that multiple users want to access at the same time. That is why the system shall be able to handle simultaneous data in- and output.
The website shall be able to handle up to 20 users at the same time, which could be scaled up if necessary.

2.4.4 Reliability requirements

The website shall be operational if the internet connection can be established successfully.

The application should be finished on the 29th of December if possible and shall be finished on the 12th of January at latest.

2.4.5 Criticality of the application

AppComm depends partially on the system to communicate to their customers and manage them effectively, which makes failure somewhat business critical.
Short-term disruptions are not critical for the customer and can be temporarily replaced by manually looking up the necessary information in the database. But every disruption will lead to a possible waste of time for AppComm employees and customers, so any reason for disruptions should be removed as soon as the disruptions occur.

2.4.6 Safety and security considerations

Only AppComm employees and their customers shall be able to access the website, which is ensured since the website is secured with a username and password. A customer shall be able to see an overview of his information but not the information of other customers.

2.5 Assumptions and dependencies

The operating system should be up to date and a stable internet connection shall be available.
If no internet connection is available, the website and thus the information won't be accessible and new information cannot be added to the system until an internet connection is ensured.

2.6 High-order Language and Interface Requirements

For the languages and interfaces, the following requirements were given by the customer:

- Php and the framework "Laravel" shall be used on the newest stable version available
- Inertija.js shall be used for server-side routing and control
- Vue.js shall be used for the frontend of the application
- The product shall be dockerized, to easily deploy it on other systems

2.7 Apportioning of requirements

The following subchapter cannot be answered at this point in the development:

- Memory
- Hardware limitations
- Interfaces to other applications
- Audit functions
- Control functions
- Signal-handshake protocols

Those subchapters have a lower priority for the customer, and they are not in the scope of the first release. However, they might be important in future versions of the TCT.

3 Specific requirements

This chapter gives a detailed description of the system and all its features. Furthermore, all the functional and quality requirements the system contains are listed and explained.

3.1 External interfaces

This section provides basic prototypes of the user interface.

3.1.1 User Interfaces

On the following pages, the website wireframes will be shown to visualize the designers and the customers ideas. These wireframes were designed considering subchapter 2.1.2 and NFR4 as well as NFR7.

After logging in, on the home screen, the customer should be greeted and could receive some information.
In the customer overview for the administrator, there should be a table with some entries. For each column, filtering must be possible. Furthermore, the user needs to be able to search for entries or create new ones. In the top, returning to the overview or to the homepage shall always be possible.

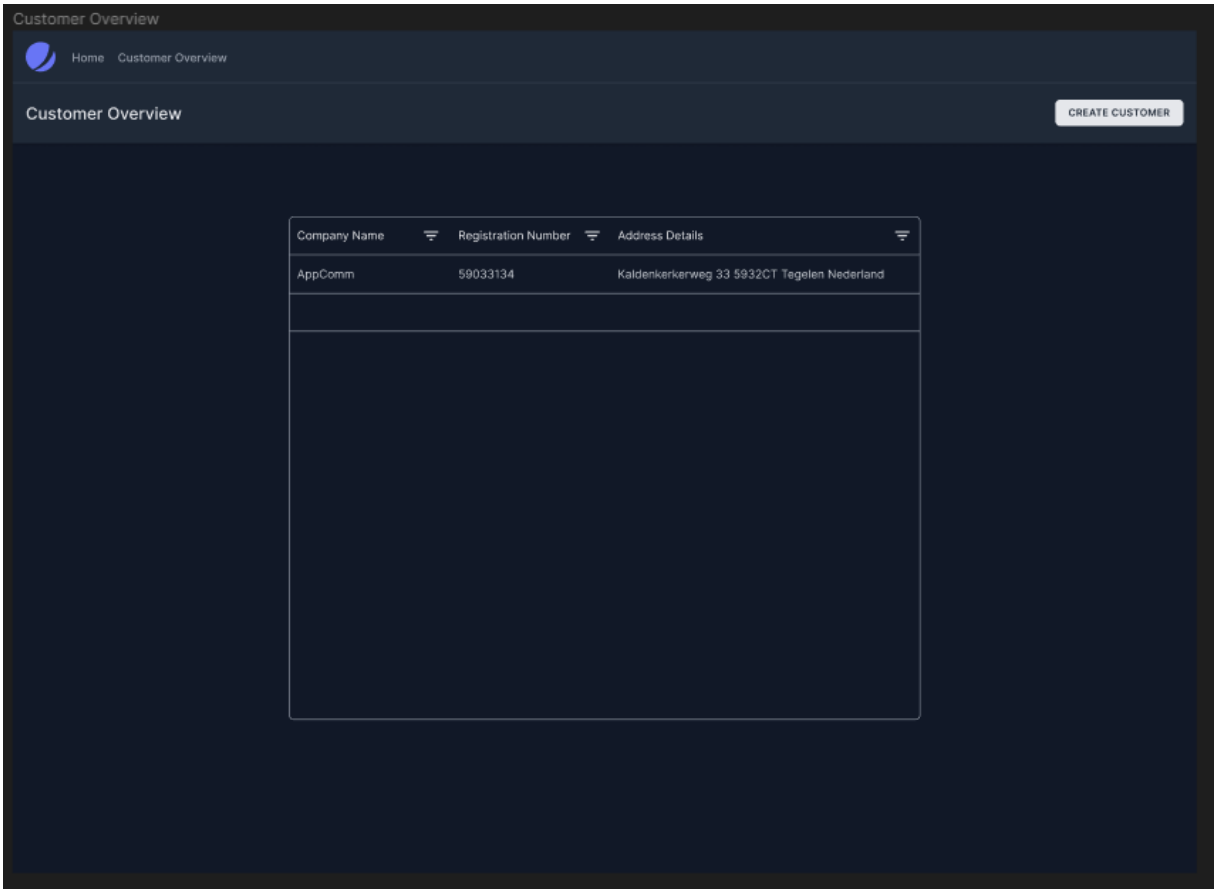


FIGURE 2 CUSTOMER OVERVIEW

When creating a new customer, it shall be clear which information belongs to each field.
Furthermore, it shall be visible which fields are required and which ones optional.

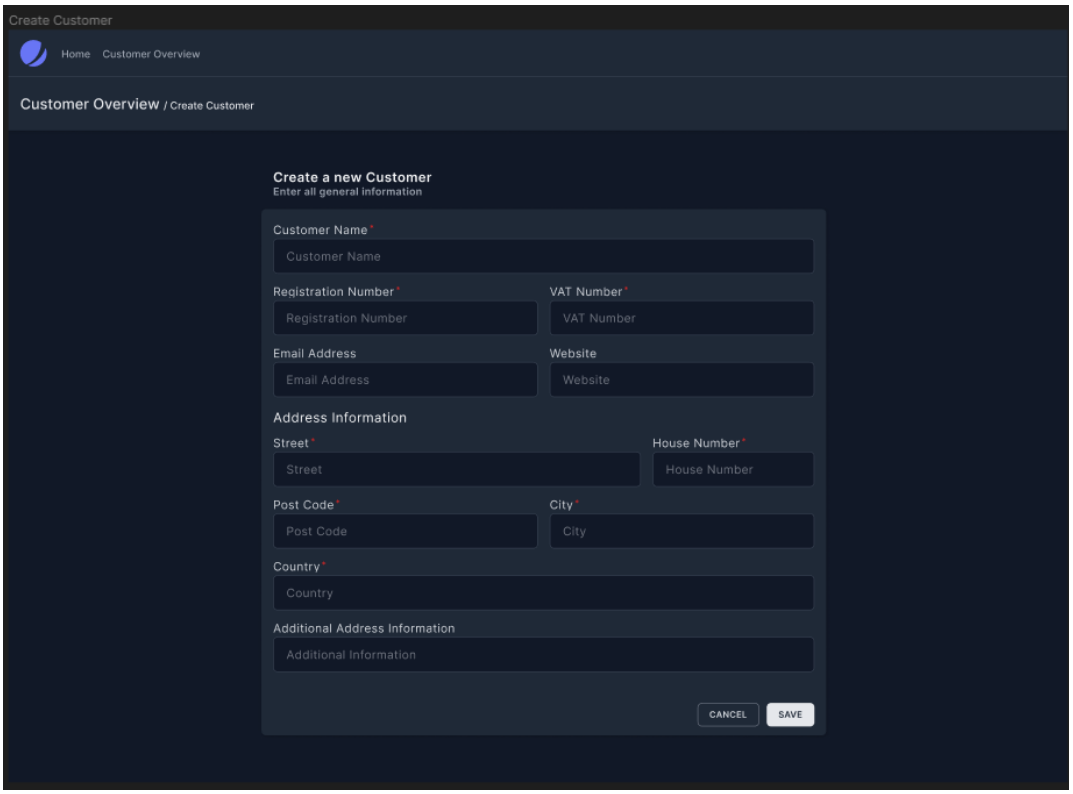


FIGURE 3 CREATE CUSTOMER

In the overview, the customer shall have a division between the general company information, the address information, the contact persons, and the solution.

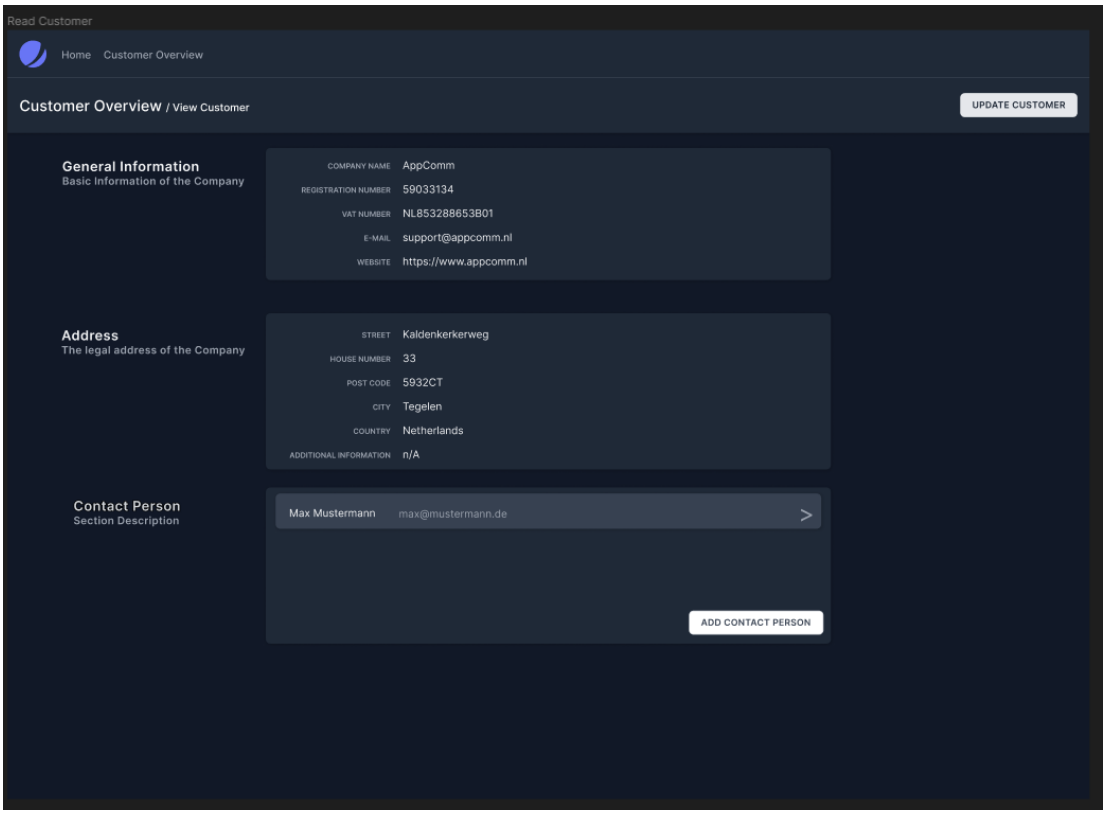


FIGURE 4 READ CUSTOMER

When updating the customer, all fields shall become editable, and it shall be possible to (de)activated or delete a customer from here.

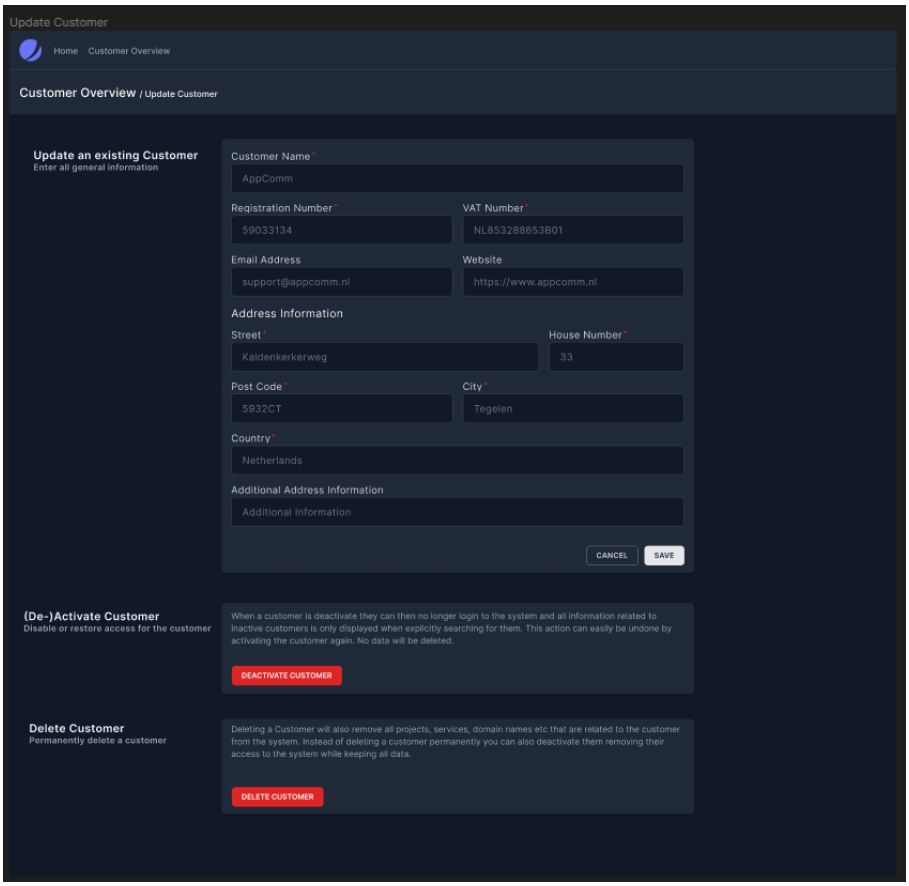


FIGURE 5 UPDATE CUSTOMER

For the deletion, an administrator shall always get asked to confirm this non-undoable action.

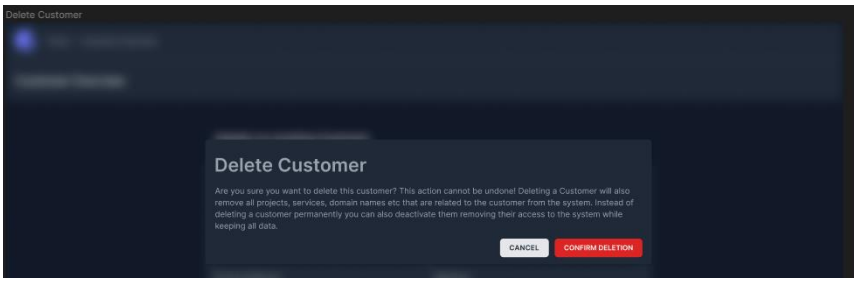


FIGURE 6 DELETE CUSTOMER

For the (de)activation, an administrator shall always get asked to confirm this undoable action.

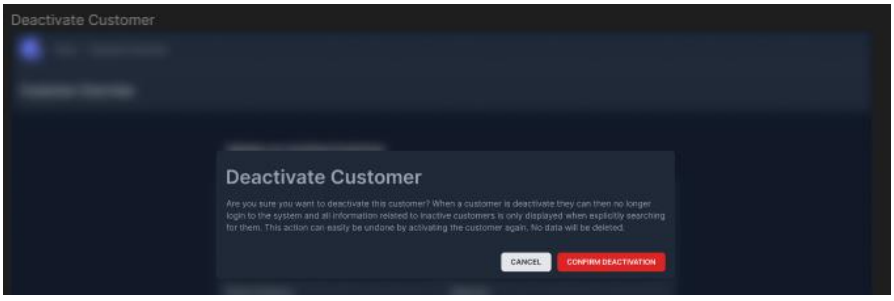


FIGURE 7 (DE)ACTIVATE CUSTOMER

When creating a new contact person, the customer name should be visible to avoid confusion. There should be general information about the person and specific contact information.

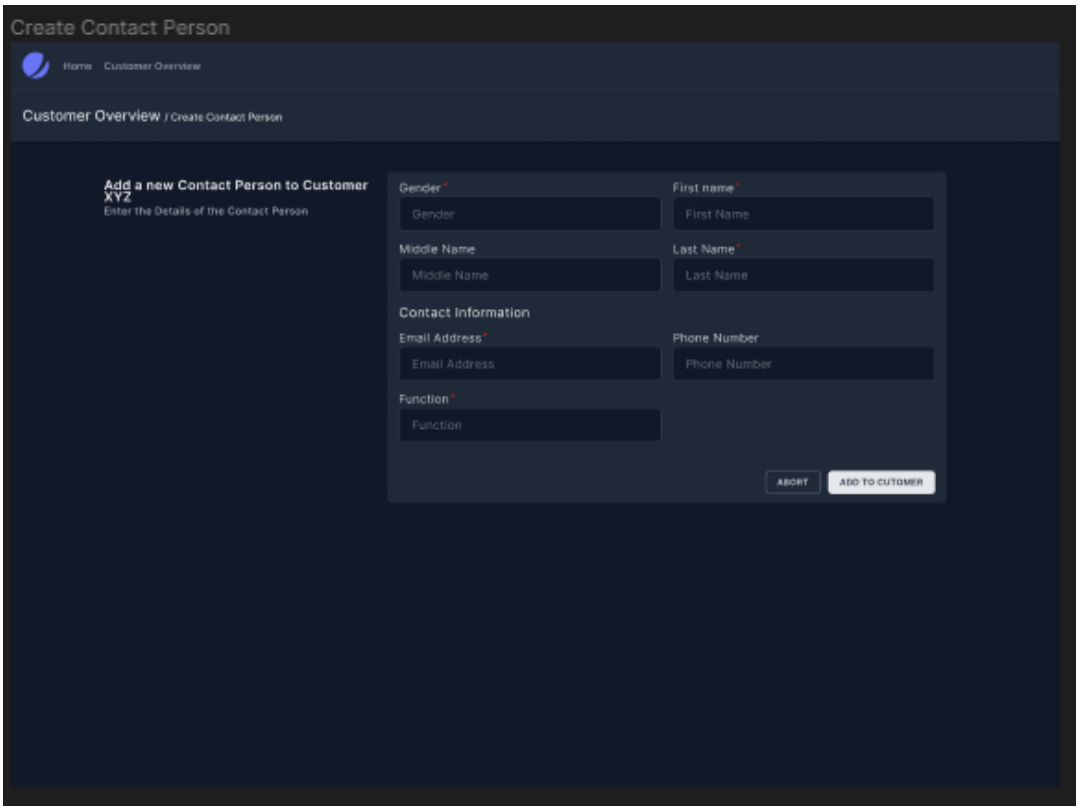


FIGURE 8 CREATE CONTACT PERSON

Just like for the creation, while reading a contact person, the customer's name should be visible.

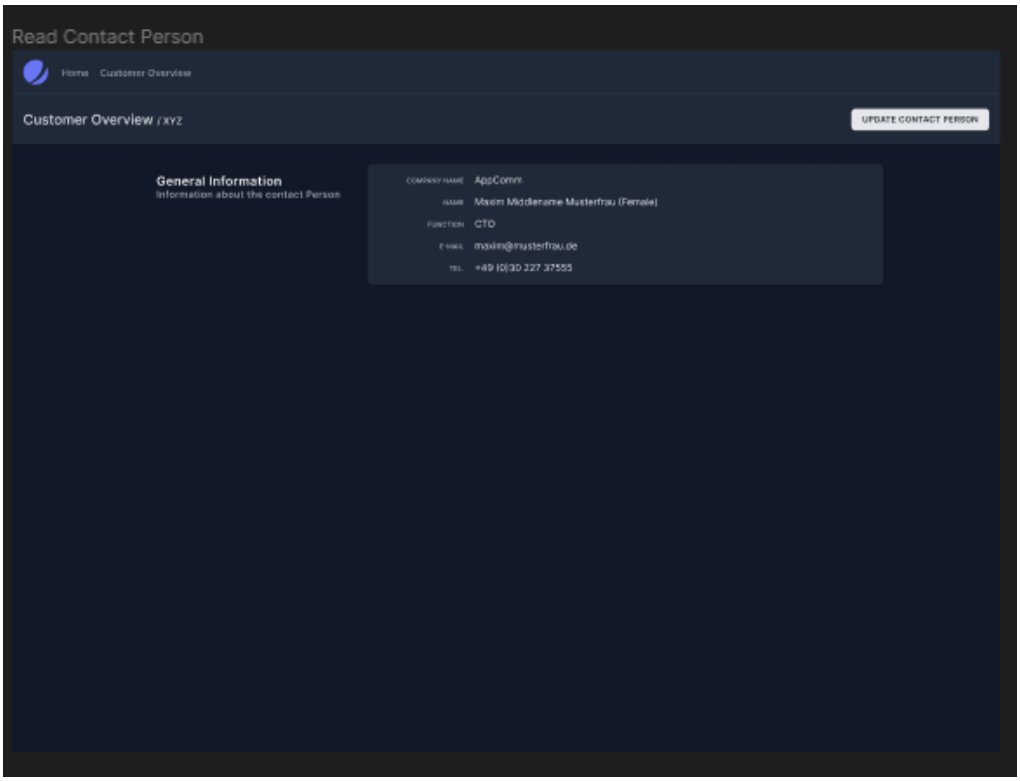


FIGURE 9 READ CONTACT PERSON

For the update, just like for the customer itself, the delete option shall be found here.

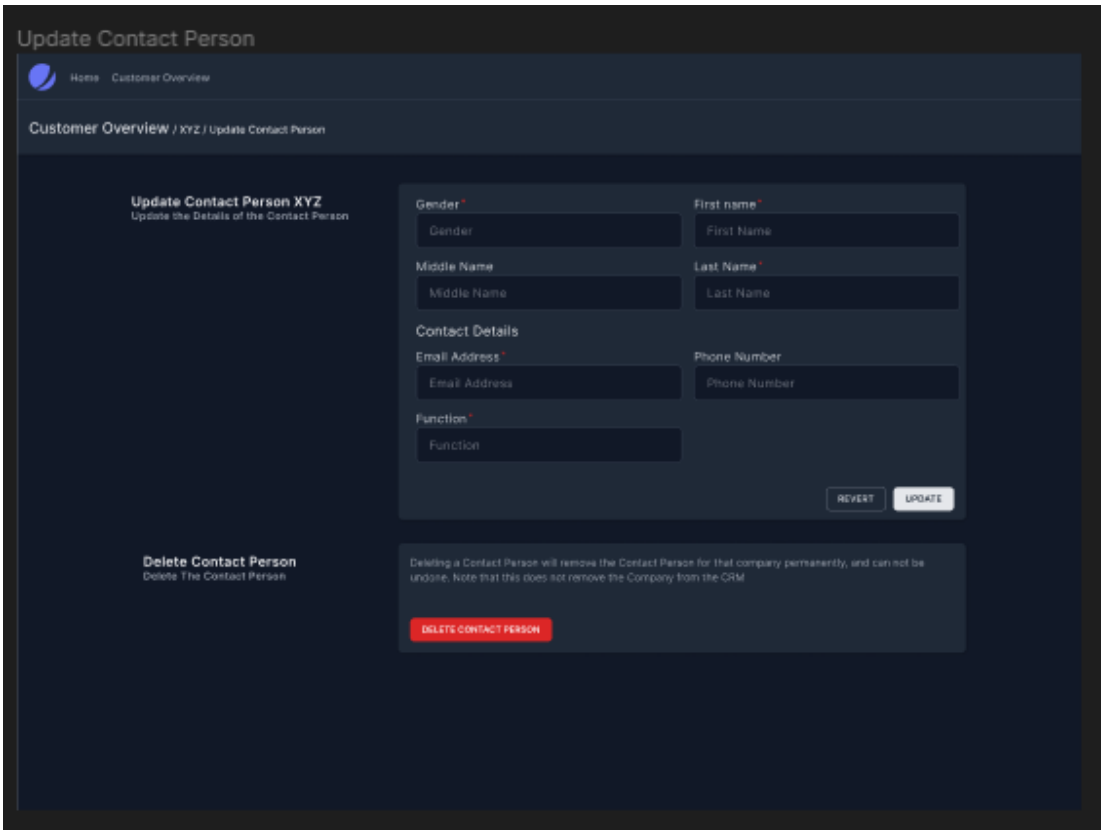


FIGURE 10 UPDATE CONTACT PERSON

If the contact person or any other entry gets updated, there could be a window to confirm this.

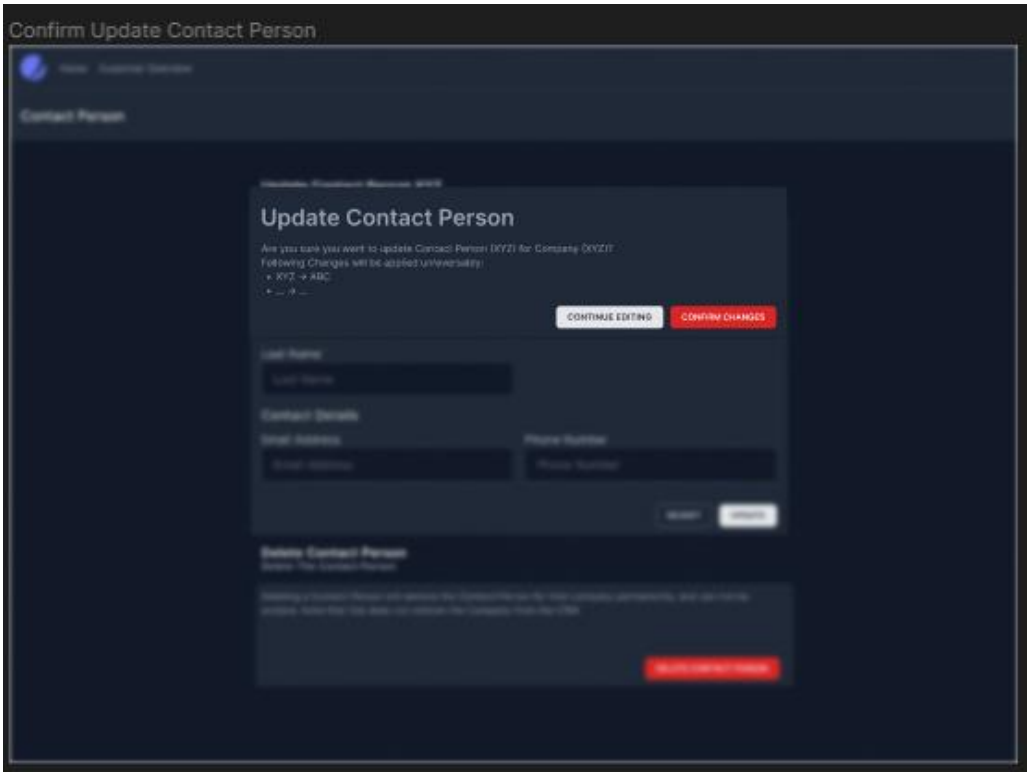


FIGURE 11 CONFIRM UPDATE CONTACT PERSON

For the deletion, there shall be the same window as for the customer deletion.

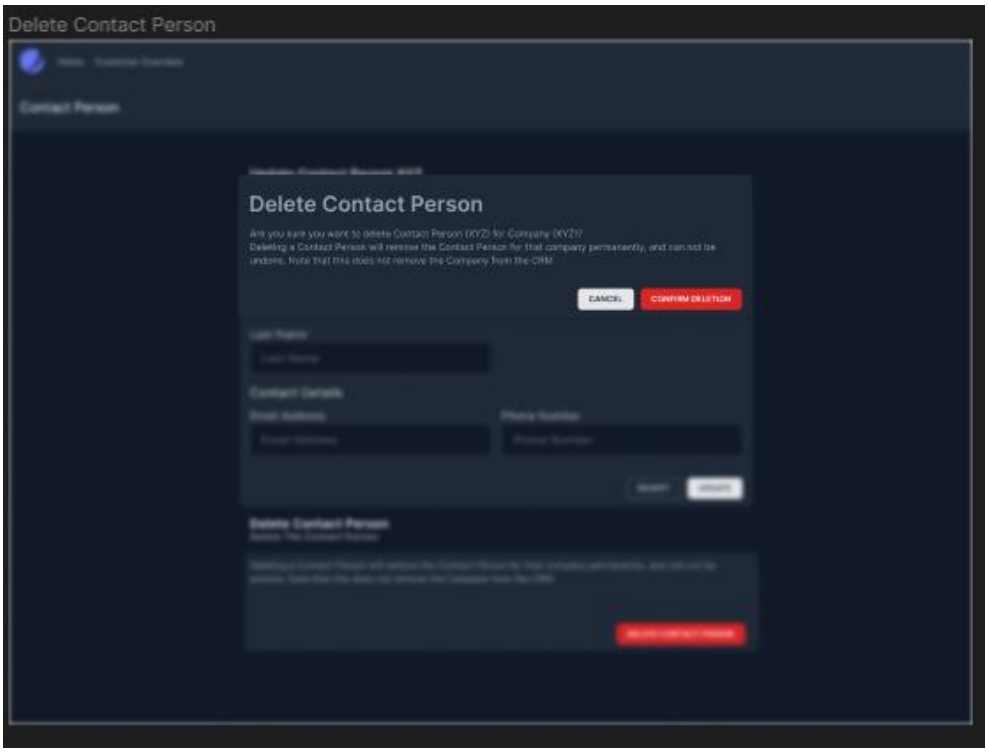


FIGURE 12 CONFIRM DELETE CONTACT PERSON

Just like for the customer, there shall also be a solution overview for all the available solutions.

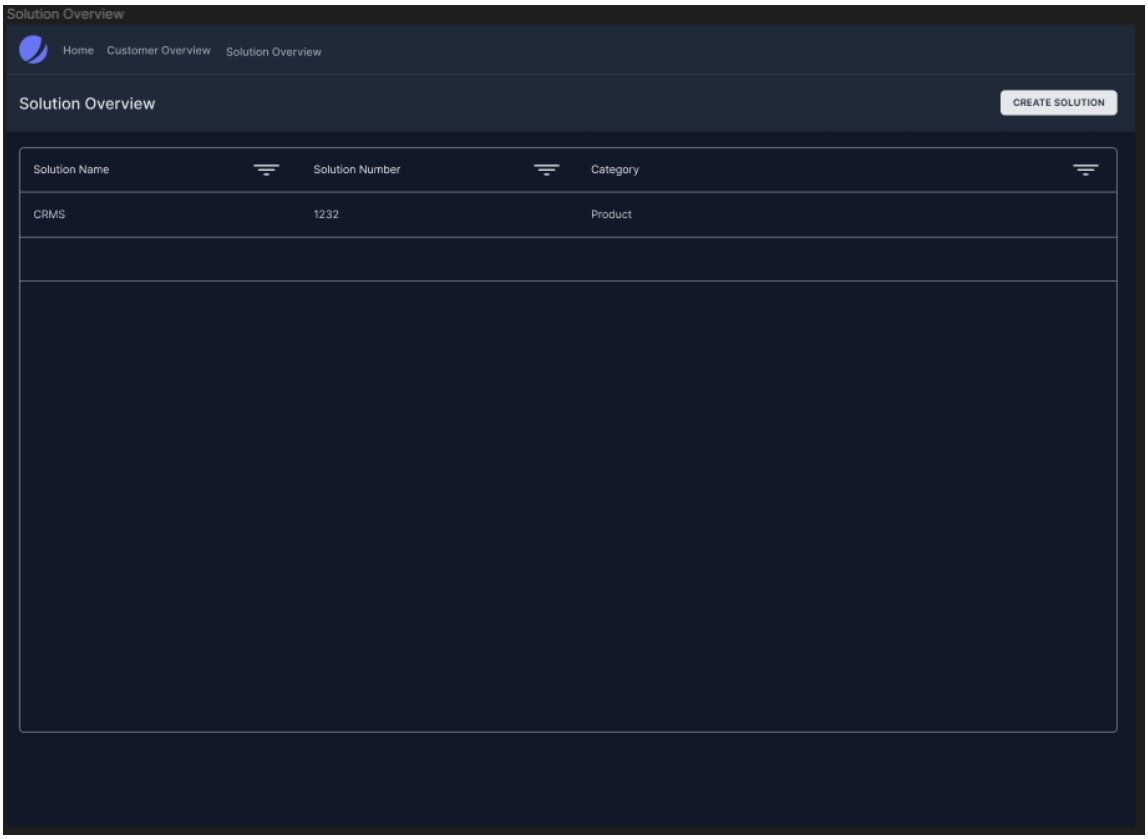


FIGURE 13 SOLUTION OVERVIEW

When creating a solution, the administrator will be able to see that except the description, all fields need to be filled out.

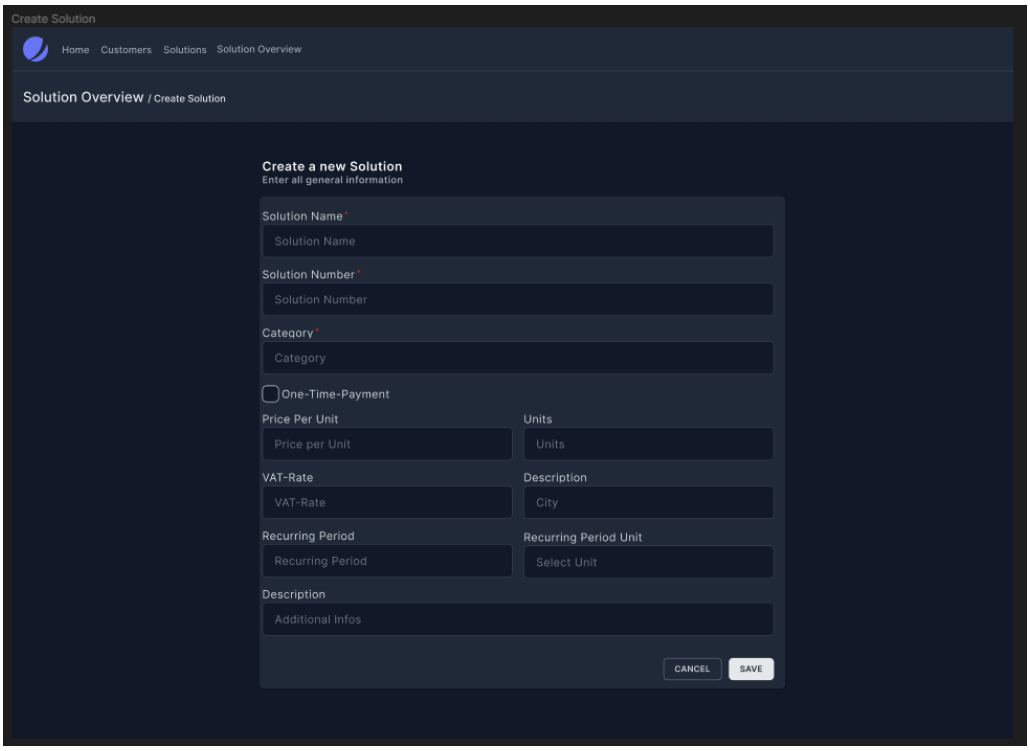


FIGURE 14 CREATE SOLUTION

Since the description might be a bit longer, it won't be shown on the overview page of all solutions, but it can be seen on the specific overview page.

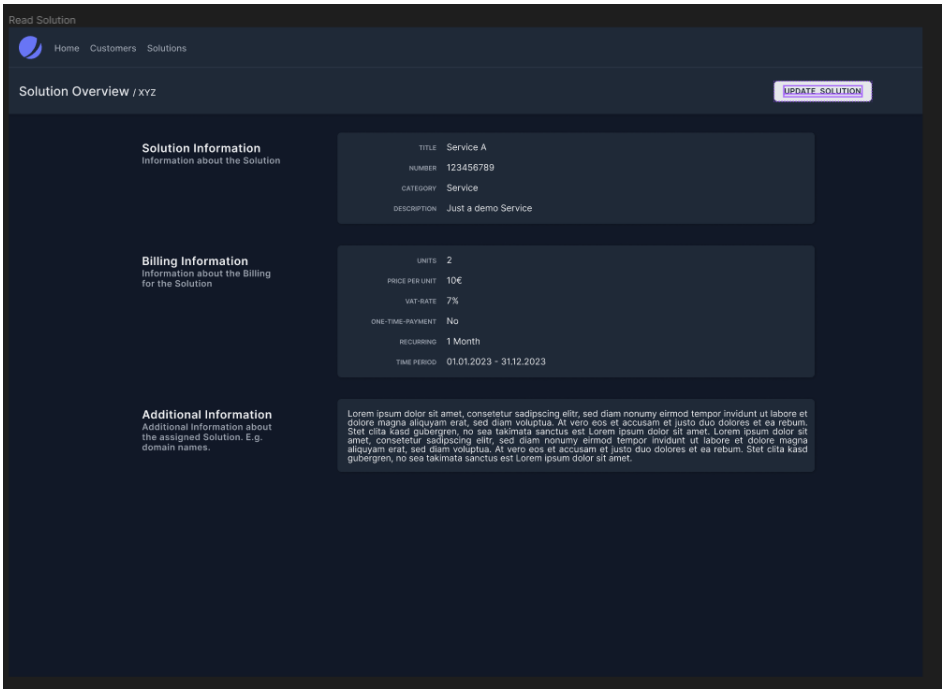


FIGURE 15 READ SOLUTION

For the update, it should look similar to the creation of the solution.

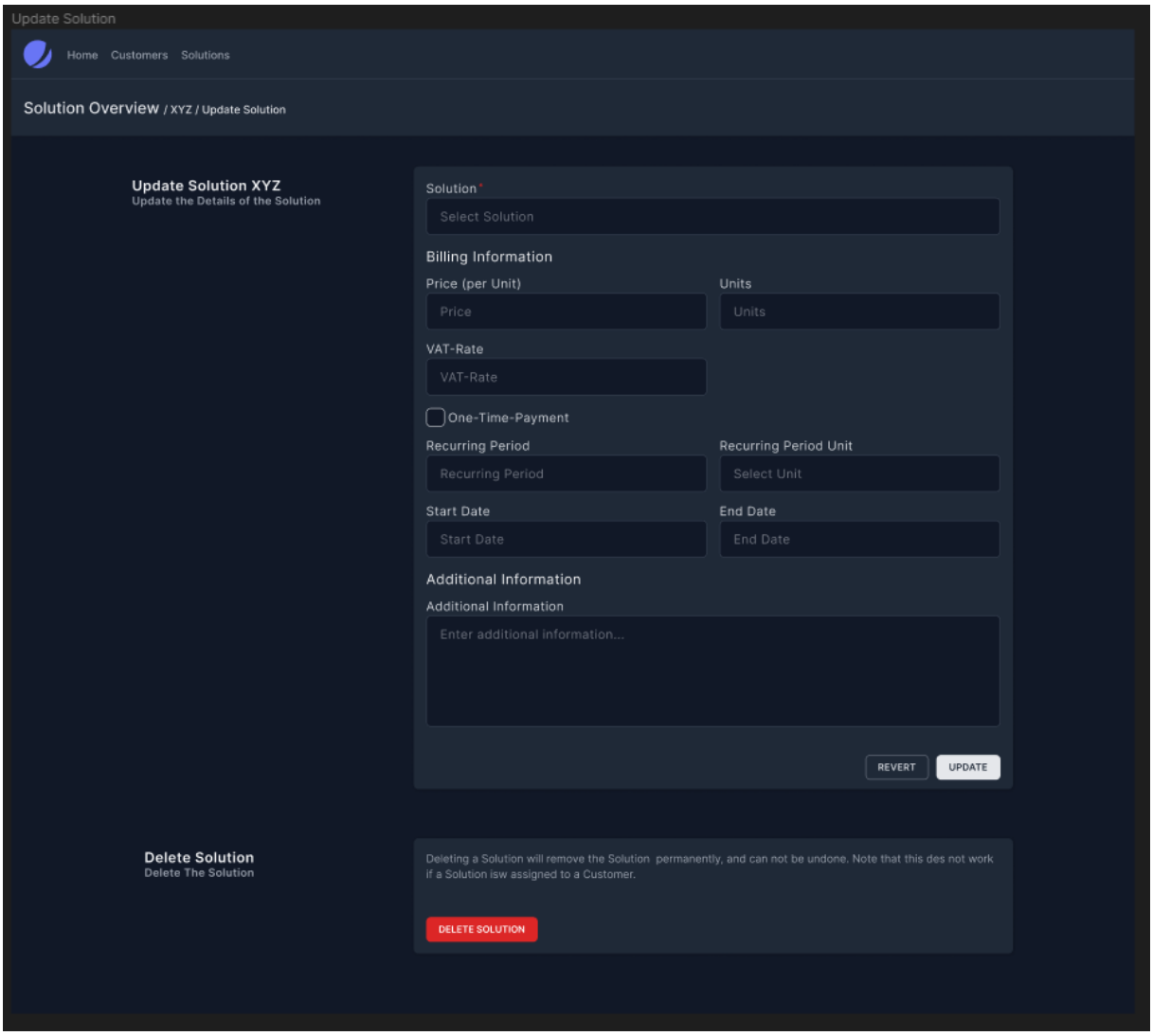


FIGURE 16 UPDATE SOLUTION

When the administrator wants to delete the solution, he shall be informed that it can’t be undone.

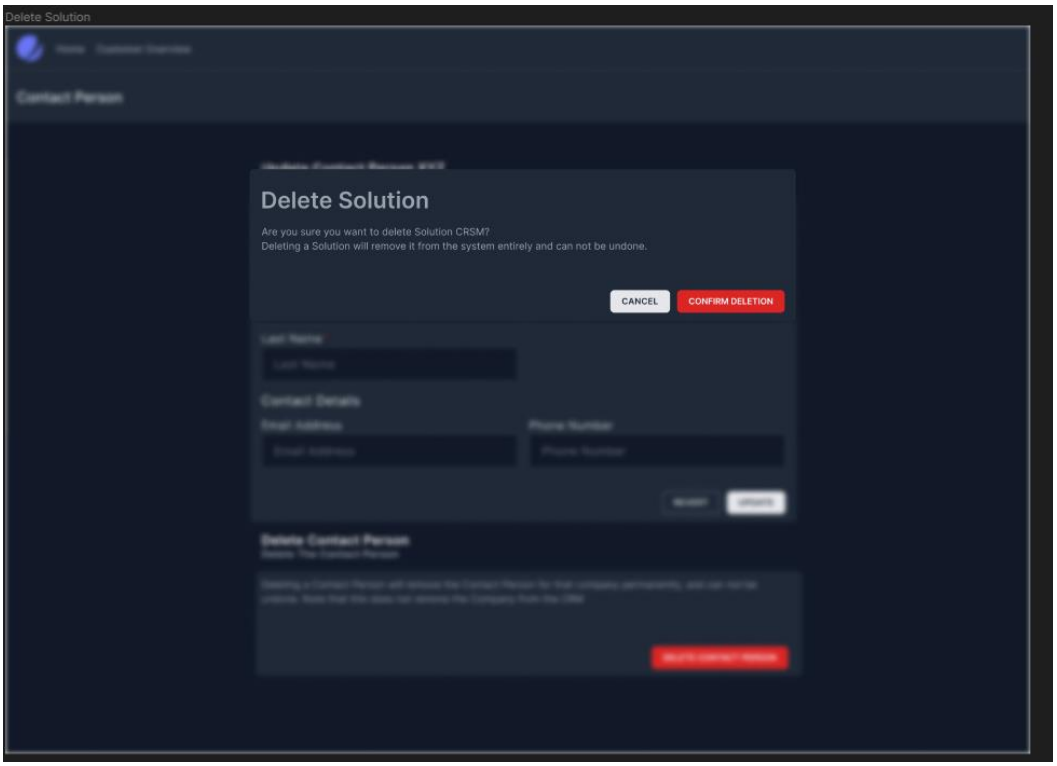


FIGURE 17 DELETE SOLUTION

If there is any kind of issue with a solution, a administrator can create a ticket for one of the already existing GitLab projects in a specific solution .

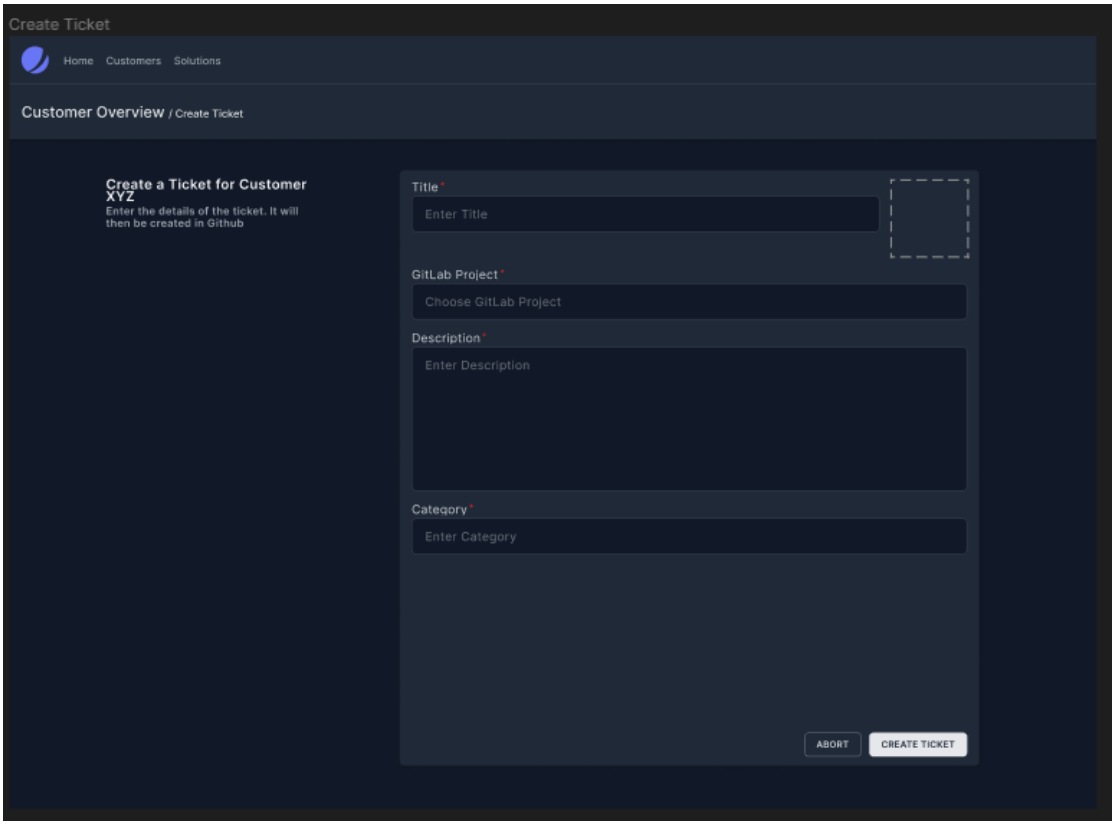


FIGURE 18 CREATE TICKET

3.2 Functions

This section includes all relevant requirements that specify the actions of the software system.

3.2.1 Requirements and their relationship towards the Use Cases

In the following table, an overview is given to show which Use Case fulfils which functional requirement.

Category	Use Case	Functional Requirement
“Customer Use Cases”	UC1.1 – Create Customer	FR1, FR2, FR5
“Customer Use Cases”	UC1.2 – Read Customer	FR10, FR11
“Customer Use Cases”	UC1.3 – Update Customer	FR1, FR2, FR8
“Customer Use Cases”	UC1.4 – Delete Customer	FR15, FR17
“Customer Use Cases”	UC1.5 – Deactivate Customer	FR13, FR16
“Customer Use Cases”	UC1.6 - Activate Customer	FR13, FR16
“Customer Use Cases”	UC1.7 - View Customer Overview	FR10, FR11, FR12
“Solution Use Cases”	UC2.1 – Create Solution	FR1, FR2, FR5
“Solution Use Cases”	UC2.2 – Read Solution	FR10, FR11
“Solution Use Cases”	UC2.3 – Update Solution	FR1, FR2, FR8
“Solution Use Cases”	UC2.4 – Delete Solution	FR15, FR18
“Solution Use Cases”	UC2.5 – View Solution Overview	FR10, FR11, FR12
“Solution Use Cases”	UC2.6 – Assign Solution	FR2, FR3, FR4, FR5
“Solution Use Cases”	UC2.7 Unassign Solution	FR16
“Contact Person Use Cases”	UC3.1 – Create Contact Person	FR1, FR2, FR5
“Contact Person Use Cases”	UC3.2 – Read Contact Person	FR10, FR11
“Contact Person Use Cases”	UC3.3 – Update Contact Person	FR1, FR2, FR8
“Contact Person Use Cases”	UC3.4 – Delete Contact Person	FR15
“Ticket Use Cases”	UC4.1 Assign GitLab Project	FR2, FR5
“Ticket Use Cases”	UC4.2 Unassign GitLab Project	FR16
“Ticket Use Cases”	UC4.3 Write Ticket	-

TABLE 4 RELATIONSHIP BETWEEN THE FUNCTIONAL REQUIREMENTS AND THE USE CASES

3.2.2 Enter new information

3.2.2.1 Functional Requirement 1.1

ID: FR1

TITLE: Duplicate entries

DESC: Duplicate entries for the same entry shall not be allowed.

RAT: Otherwise, data integrity cannot be guaranteed.

DEP: FR2 (Feedback on duplicate entry), FR8 (Changes lead to no duplicate entries)

3.2.2.3 Functional Requirement 1.2

ID: FR2

TITLE: Pointing out errors

DESC: The person filling in the entry shall always get feedback on which errors were made during the input of data.

RAT: To ensure that the user understand the issue and can correct it.

DEP: FR3 (Correctly filling out fields), FR5 (Empty required fields pointed out)

3.2.2.4 Functional Requirement 1.3

ID: FR3

TITLE: None or all price fields filled

DESC: The price of a solution is optional, but if it is added, all required information about the price shall be present.

RAT: Either missing or fully working, no half-way options.

DEP: FR4 (Use default prices where applicable)

3.2.2.5 Functional Requirement 1.4

ID: FR4

TITLE: Default Price in Solution

DESC: If the assigned solution doesn't get a new "price", the default price shall be used.

RAT: The price for the solution should have a default value.

DEP: None

3.2.2.6 Functional Requirement 1.5

ID: FR5

TITLE: "Missing data" entries not accepted

DESC: Entries with missing "required" fields shall never be successfully added to the database.

RAT: Data integrity shall be guaranteed.

DEP: None

3.2.2.7 Functional Requirement 1.6

ID: FR6

TITLE: Track the altering of information

DESC: For every new entry or change of an entry, the user who did so should be stored.

RAT: To track who did what if there arise any problems

DEP: None

3.2.2.8 Functional Requirement 1.7

ID: FR7

TITLE: Quick note feature

DESC: Additional to the information of the customer, the administrator could be able to create a "quick note feature" on the specific customer page. [Not implemented yet.]

RAT: For quick notes during a talk with the customer.

DEP: None

3.2.3 Change existing information

3.2.3.1 Functional Requirement 2.1

ID: FR8

TITLE: Change of information leads to no issues

DESC: When changing existing information, the data integrity of the entry shall be ensured.

RAT: Data integrity is really important when it comes to an application for information.

DEP: FR9

3.2.3.1 Functional Requirement 2.2

ID: FR9

TITLE: Check changes

DESC: After changing, the user could be asked if the changes are correct.

RAT: Data integrity is really important when it comes to an application for information.

DEP: FR8

3.2.4 View information

3.2.4.1 Functional Requirement 3.1

ID: FR10

TITLE: All entries present

DESC: The entry not only contains the main information about the customer, solution or else but also each sub-information created for the specific entry.

RAT: Data integrity needs to be ensured.

DEP: None

3.2.4.2 Functional Requirement 3.2

ID: FR11

TITLE: Overview up to date

DESC: All pages within the application shall use the newest database-status. Refreshing the application shall lead to an update of the data, if necessary.

RAT: Latest information shall be included to ensure the integrity of data.

DEP: Internet connection

3.2.4.3 Functional Requirement 3.3

ID: FR12

TITLE: Filtering entries based on criteria possible

DESC: For each column of information in the overview table it shall be possible to filter this column.

RAT: If there are many rows, it is necessary to only see the relevant rows. Furthermore, deleting certain rows gets way easier by doing so.

DEP: None

3.2.4.4 Functional Requirement 3.4

ID: FR13

TITLE: Deactivated customers

DESC: When deactivated, customer shall not be able to see any information.

RAT: If a customer is deactivated, this customer is not allowed to view any of AppComm's information.

DEP: None

3.2.4.5 Functional Requirement 3.5

ID: FR14

TITLE: Progress overview

DESC: The status of a solution could be visible to both the administrator as well as the customer when viewing a specific Solution. [Not implemented yet.]

RAT: Helpful overview for both the customer as well as the administrator.

DEP: None

3.2.5 Deleting information

3.2.5.1 Functional Requirement 4.1

ID: F15

TITLE: Confirmation of non-undoable actions

DESC: For deleting a customer, the administrator shall always have to confirm this deletion, which is not undoable.

RAT: Avoid errors

DEP: None

3.2.5.2 Functional Requirement 4.2

ID: FR16

TITLE: Confirmation of undoable actions

DESC: For activating and deactivating a customer, the administrator shall always have to confirm this deletion, which is undoable.

RAT: Avoid errors

DEP: None

3.2.5.3 Functional Requirement 4.3

ID: FR17

TITLE: Data integrity when deleting

DESC: All address data as well as contact persons shall be deleted when the respective customer is deleted.

RAT: Avoid errors

DEP: 10

3.2.5.4 Functional Requirement 4.4

ID: FR18

TITLE: Deleting solution only when not assigned

DESC: Deleting a solution shall only work if it is not assigned to any customer.

RAT: Avoid errors and ensure data integrity

DEP: 10

3.3 Performance requirements

3.3.1 Search feature

ID: NFR1

TITLE: Simple search and filter feature

DESC: The search and filter functionality and the displayed entries should be easy to use and understand. A first-time user should not take more than 2 minutes searching for an entry and accessing it, if found.

RAT: Not as important as stability of the system, still quite important for users, especially if they are not used to accessing entries through a website.

DEP: None

3.3.2 Response time: User input

ID: NFR2

TITLE: Fast UI features

DESC: UI elements that do not involve outside connections should respond to user input immediately with no noticeable waiting time.

RAT: For a fast interaction with the system

DEP: Processor of device

3.3.3 Fast updates

ID: NFR3

TITLE: Fast update feature

DESC: The user could be able to refresh the page to get the most recent status of the database displayed in the application.

RAT: Relevant so changes of information are displayed both in tables as well as in dashboards.

DEP: Processor of device

3.3.4 System stability: Simultaneous users

ID: NFR4

TITLE: Number of simultaneous users

DESC: The system could be designed to handle up to 20 users simultaneously. This number can potentially be increased in further updates.

RAT: There might be multiple people wanting to access the application at the same time.

DEP: None

3.3.5 Response time: Search function

ID: NFR5

TITLE: Fast search feature

DESC: The user could not wait for the website loading longer than 5 seconds in 95% of the cases.

RAT: Necessary for a fast search

DEP: Processor of the device

3.3.6 System stability: Simultaneous entries

ID: NFR6

TITLE: Number of simultaneous actions

DESC: The system could be able to handle up to 10 actions such as creating or changing entries.

RAT: There might be multiple people wanting to use the application at the same time.

DEP: None

3.3.7 User Experience

ID: NFR7

TITLE: As little steps as possible

DESC: The number of steps required for the user to reach a specific page shall be minimized.

RAT: Time is a really important factor in the success of the application, so a fast “creation time” is necessary.

DEP: None

3.4 Logical database requirements

The important entities of the “Chat Robot Project” relating the database are the customer, the contact person and the solution. Furthermore, “address information” and “ticket” are relevant as well. Those entities are created and possibly changed by administrators, while customers are restricted to creating and altering entries only for themselves and not for other customers. The relationship between those entities can be seen in the domain model.

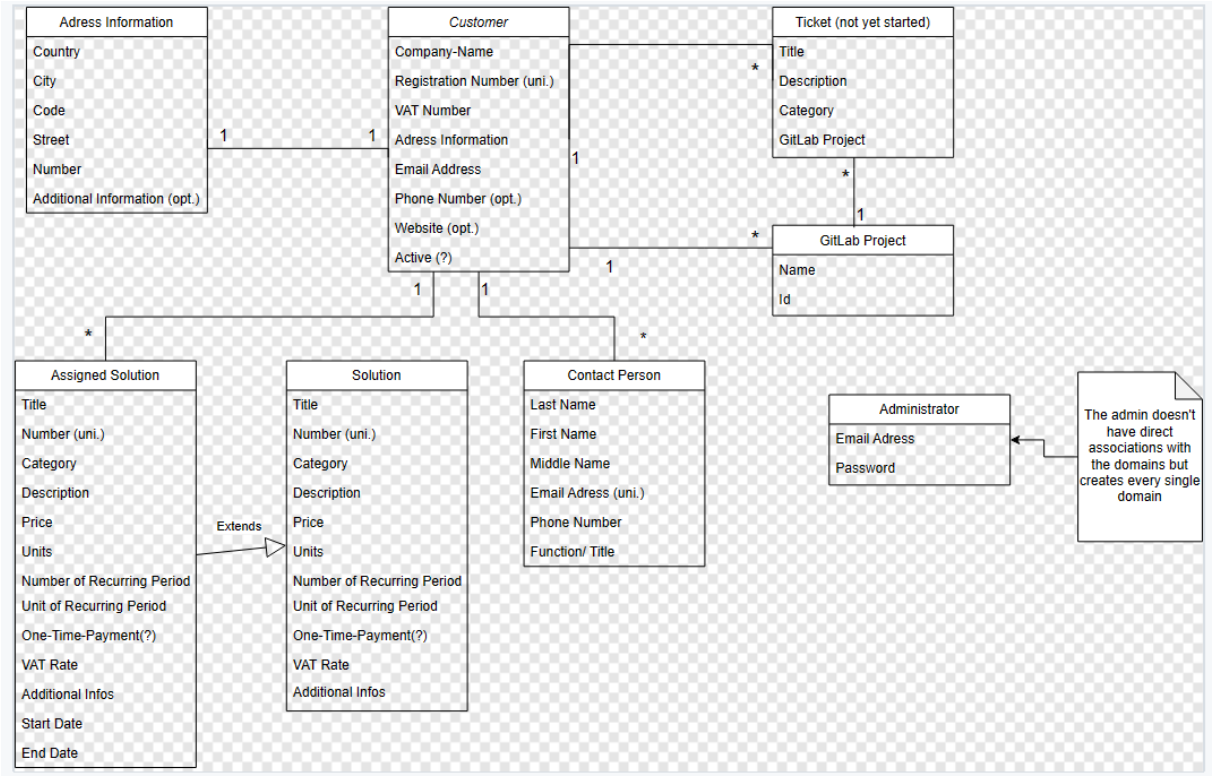


FIGURE 19 DOMAIN MODEL

Each Customer has exactly one “address information” for themselves. A customer can have 0 to multiple assigned solutions and contact persons, but both assigned solutions and contact persons can’t exist without a customer. Those assigned solution extend the “normal” solution, since multiple customers can have the same solution with different conditions. A customer can create 0 to many tickets and each ticket can be worked on and answered by MULTIPLE administrators. So far, since there are no different users in the system, it is also the admin who creates the ticket.

3.5 Design constraints

3.5.1 Memory Usage

ID: NFR8

TITLE: Memory usage of the working software

DESC: The application should use no more than 50 Megabyte of memory, preferable less, for the mobile version

RAT: So that the website is capable of running on every device

DEP: Operating system of the mobile device

3.6 Software system attributes

3.6.1 Reliability

ID: NFR9

TITLE: Reliability of the system

DESC: The application shall provide the right result in search 100% of the time. Searching for an entry shall in every case lead to the entry being found if present. Furthermore, data integrity shall be guaranteed at all times.

RAT: In order for a perfectly reliable application, especially data integrity is really important.

DEP: None

3.6.2 Availability

ID: NFR10

TITLE: Stability of the service

DESC: The website shall be accessible in 100% of the attempts when the device is connected to the internet and the ping is lower than 100ms. Downtimes of the app should only take place if there are major technical issues.

RAT: Most important, since the user needs to be able to access the website to interact with it

DEP: A working webserver and database is required.

3.6.3 Security

ID: NFR11

TITLE: Security - Privacy

DESC: Only relevant data is stored. The stored information is for AppComm only and shall not be given to any third-party. In case of a data leak, all involved entities shall be immediately informed about the situation.

RAT: Keeping information regarding companies and their private information safe and secure.

DEP: GDPR

3.6.4 Maintainability

ID: NFR12

TITLE: Maintainable and extendable website

DESC: The customer has ideas to extend the application by adding the “Chat Robot” as a data source, as mentioned in 2.4.3. The implemented project shall be documented, and this document shall be readable for everyone to make it as maintainable as possible.

RAT: High maintainability is necessary if the product needs to be changed, updated, or even fixed.

DEP: None

3.6.5 Portability

ID: NFR13

TITLE: Application portability

DESC: All devices (mobile and desktop) which are used within the company shall be able to access the website and use every feature.

RAT: In order to run on every system and device.

DEP: None

3.7 Organising the specific requirements

As mentioned above, the functional requirements are divided into “Categories”. All requirements in 3.2 are grouped by their functionality and both the requirements in 3.2 and 3.3 are ranked for importance.

3.8 Additional comments

Multiple chapters and subchapters were left out of the SRS, at example “Index”. Those chapters haven’t been forgotten but were simply not applicable in the project and thus weren’t written down here. The SRS was written based on the IEEE-Template, so that all developers (and other people who read this report) directly know the structure and where to find the relevant parts for them. The only thing which differs from the common SRS-Reports is the “Testing Section” within the Appendix. In this section the test scenarios for the use cases can be found. Other than that, the structure of the SRS as well as the conventions “Shall, Should, Could” are all based on the IEEE-Template.

3.9 Version History

Version	Date	Main Updates
Version 1.0	22.09.	First Version, mainly the structure and Key Requirements
Version 1.1	06.10.	Customer User Stories and their Use Cases, as well as most of “Chapter 2” were added
Version 1.2	20.10.	“Contact Person” User Stories were added, minor updates
Version 1.3	03.11.	“Solution” User Stories added, large additions in regards to Requirements and Domain Model
Version 2.0	24.11.	Updated Structure, Updated the ranking of the Requirements based on the customers prioritisation
Version 2.1	15.12.	“Ticket” User Stories added
Version 2.2 (Final)	14.01.	Finished SRS, small adjustments

TABLE 5 VERSION HISTORY

Appendixes

(The Links work for readers authorised to access the GitHub Repository)

Test Scenarios

Create Customer

Name:	**Administrator creates customer with missing information**
Scenario:	1. Administrator indicates that a new customer needs to be added. 2. System ask for general customer information: Company (Name), Registration Number(unique), VAT Number, address (Country, City, ZIP Code, Street, number, additional address information(optional)), general mail address, phone number (optional), website(optional). 3. Administrator fills in the following information: "Company X" as the name, "123456789" as the Registration number, "Netherlands, Venlo, 11245, Hagelseweg, 12a, 1. Floor" for the address, "companyx@web.com" as mail address, leaves the optional phone number and website out and indicates to save the customer. 4. System indicates that the required field "VAT Number" is empty.
Results:	Customer creation failed.

Name:	**Administrator creates customer with incorrect data**
Scenario:	1. Administrator indicates that a new customer needs to be added. 2. System ask for general customer information: Company (Name), Registration Number(unique), VAT Number, address (Country, City, ZIP Code, Street, number, additional address information(optional)), general mail address, phone number (optional), website(optional). 3. Administrator fills in the following information: "Company X" as the name, "NL123456789B1234" as VAT number, "123456789" as the Registration number, "Netherlands, Venlo, 11245, Hagelseweg, 12a, 1. Floor", for the address, "companyx@web.com" as mail address, leaves the optional phone number and website out and indicates to save the customer. 4. System indicates that the VAT number is not valid.

Name: **Administrator creates customer with incorrect data**

Results: Customer creation failed.

Name: **Administrator creates customer who already exists**

Scenario: 1. Administrator indicates that a new customer needs to be added.

2. System ask for general customer information: Company (Name), Registration Number(unique), VAT Number, address (Country, City, ZIP Code, Street, number, additional address information(optional)), general mail address, phone number (optional), website(optional).

3. Administrator fills in the following information: "Company X" as the name, "NL123456789B12" as VAT number, "123456789" as the Registration number, "Netherlands, Venlo, 11245, Hagelseweg, 12a, 1. Floor", for the address, "companyx@web.com" as mail address, leaves the optional phone number and website out and indicates to save the customer.

4. System indicates that the customer is already present in the database.

Results: Customer creation failed.

Name: **Administrator successfully creates customer**

Scenario: 1. Administrator indicates that a new customer needs to be added.

2. System ask for general customer information: Company (Name), Registration Number(unique), VAT Number, address (Country, City, ZIP Code, Street, number, additional address information(optional)), general mail address, phone number (optional), website(optional).

3. Administrator fills in the following information: "Company X" as the name, "NL123456789B12" as VAT number, "123456789" as the Registration number, "Netherlands, Venlo, 11245, Hagelseweg, 12a, 1. Floor", for the address, "companyx@web.com" as mail address, leaves the optional phone number and website out and indicates to save the customer.

4. System indicates that the customer got successfully added to the database.

Results: Customer creation successful.

Read Customer

Name:	**Administrator wants to view a specific customer**
Scenario:	1. System displays the overview page.
	2. Administrator clicks on "Company X".
	3. System opens the page of "Company X" and displays the information to the administrator.
Results:	Administrator can successfully see a customers information

Update Customer

Name:	**Updating customer with missing data**
Scenario:	1. Administrator indicates that the customer information need to be updated
	2. System makes the fields editable
	3. Administrator deletes the customer name and indicates the system to save the changes.
	4. System indicates that data is missing in certain fields.
Results:	The customer's information were not altered successfully.

Name:	**Updating customer with already present data**
Scenario:	1. Administrator indicates that the customer information need to be updated
	2. System makes the fields editable
	3. Administrator changes the Registration Number to "012345678" and indicates the system to save the changes.

Name:	**Updating customer with already present data**
	4. System indicates that the Registration Number is already present in the system.
Results:	The customer's information were not altered successfully.

Name:	**Updating customer with incorrect data**
Scenario:	1. Administrator indicates that the customer information need to be updated
	2. System makes the fields editable
	3. Administrator changes the mail address from "albert.i@drago.nl" to "1234242424121" and indicates the system to save the changes.
	4. System indicates that the customer's mail address is incorrect.
Results:	The customer's information were not altered successfully.

Name:	**Administrator successfully updates customer**
Scenario:	1. Administrator indicates that the customer information need to be updated
	2. System makes the fields editable
	3. Administrator changes the email address from "albert.i@drago.nl" to "albert.ib@drago.nl" and indicates the system to save the changes.
	4. System indicates that the customer has been saved.
Results:	The customer's information were altered successfully.

Delete Customer

Name:	**Administrator denies to delete customer**
Scenario:	1.Administrator indicates that the customer "Company X" should be deleted.

Name:	**Administrator denies to delete customer**
	2. System warns the administrator by pointing out that this can't be undone.
	3. Administrator denies the deletion.
	4. System indicates that the customer has not been deleted.
Results:	Deletion of customer not successful.

Name:	**Administrator successfully deletes customer**
Scenario:	1.Administrator indicates that the customer "Company X" should be deleted.
	2. System warns the administrator by pointing out that this can't be undone.
	3. Administrator confirms the deletion.
	4. System indicates that the customer has been deleted, with all the information present.
Results:	Deletion of customer successful.

Deactivate Customer

Name:	**Administrator denies to deactivate customer**
Scenario:	1.Administrator indicates that the customer "Company X" should be deactivated.
	2. System warns the administrator by pointing out that this can be undone.
	3. Administrator denies the deactivation.
	4. System indicates that the customer has not been deactivated.
Results:	Deactivation of customer not successful.

Name:	**Administrator successfully deactivates customer**
Scenario:	1.Administrator indicates that the customer "Company X" should be deactivated.
	2. System warns the administrator by pointing out that this can be undone.
	3. Administrator confirms the deactivation.
	4. System indicates that the customer has been deactivated.
Results:	Deactivation of customer successful.

Activate Customer

Name:	**Administrator denies to activate customer**
Scenario:	1.Administrator indicates that the customer "Company X" should be activated.
	2. System warns the administrator by pointing out that this can be undone.
	3. Administrator denies the activation.
	4. System indicates that the customer has not been activated.
Results:	Activation of customer not successful.

Name:	**Administrator successfully activates customer**
Scenario:	1.Administrator indicates that the customer "Company X" should be activated.
	2. System warns the administrator by pointing out that this can be undone.
	3. Administrator confirms the activation.
	4. System indicates that the customer has been activated.

Name:	**Administrator successfully activates customer**
Results:	Activation of customer successful.

View Customer Overview

Name:	**Administrator views all customers**
Scenario:	1. System displays all the customers with their main information.
	2. Admin views all the information
Results:	Administrator can successfully view the companies.

Name:	**Administrator filters customer**
Scenario:	1. System displays all the customers with their main information.
	2. Admin indicates he wants to filter the customers.
	3. System gives the admin the option to filter the customers based on the fields displayed.
	4. Admin selects the location filter and only includes company from Venlo.
	5. System displays all the customers located in Venlo, which is only "Company X".
Results:	Administrator can successfully view the companies based on the filtered criteria.

Name:	**Administrator sorts customers**
Scenario:	1. System displays all the customers with their main information.
	2. Admin indicates he wants to sort the customers.

Name:	**Administrator sorts customers**
	3. System gives the admin the option to sort the customers based on the fields displayed.
	4. Admin selects the name filter and wants to sort them alphabetically.
	5. System displays all the customers in alphabetical order, starting with "Aardappelen", then "Brabandse" and then "X Factory".
Results:	Administrator can successfully view the companies based on the sorting criteria.

Create Solution

Name:	**Administrator creates solution with missing data**
Scenario:	1. Administrator indicates that a new Solution needs to be added
	2. System displays general information form, containing: number(plain number, unique), category(service/product/domain name), title, description, price (is optional, different styles, per unit/ once), VAT rate (as decimal)
	3. Administrator fills out "" as number, "product" as category, "Product A" as name, "This product is useful" as description, "1000" as price in euros, "once" as type of payment and "0.13" as VAT rate.
	4. System indicates that the number is missing.
Results:	A new Solution has not been successfully added.

Name:	**Administrator creates solution with incorrect data**
Scenario:	1. Administrator indicates that a new Solution needs to be added
	2. System displays general information form, containing: number(plain number, unique), category(service/product/domain name), title, description, price (is optional, different styles, per unit/ once), VAT rate (as decimal)

Name:	**Administrator creates solution with incorrect data**
	3. Administrator fills out "This" as number, "product" as category, "Product A" as name, "This product is useful" as description, "1000" as price in euros, "once" as type of payment and "0.13" as VAT rate.
Results:	A new Solution has not been successfully added.
Name:	**Administrator creates solution with already present data**
Scenario:	1. Administrator indicates that a new Solution needs to be added
	2. System displays general information form, containing: number(plain number, unique), category(service/product/domain name), title, description, price (is optional, different styles, per unit/ once), VAT rate (as decimal)
	3. Administrator fills out "2" as number, "product" as category, "Product A" as name, "This product is useful" as description, "1000" as price in euros, "once" as type of payment and "0.13" as VAT rate.
Results:	A new Solution has not been successfully added.
Name:	**Administrator successfully creates solution**
Scenario:	1. Administrator indicates that a new Solution needs to be added
	2. System displays general information form, containing: number(plain number, unique), category(service/product/domain name), title, description, price (is optional, different styles, per unit/ once), VAT rate (as decimal)
	3. Administrator fills out "1" as number, "product" as category, "Product A" as name, "This product is useful" as description, "1000" as price in euros, "once" as type of payment and "0.13" as VAT rate.
Results:	A new Solution has successfully been added to the database.

Read Solution

Name:	**Administrator reads specific view**
Scenario:	1. System displays the overview page of customer "Company X"
	2. Administrator clicks on the specific solution "Solution A"
	3. System opens this specific Solution's page and displays the information to the administrator.
Results:	Administrator can successfully see the overview over "Solution A".

Update Solution

Name:	**Administrator updates solution with missing data**
Scenario:	1. Administrator indicates that the Solution information need to be updated
	2. System makes the fields editable
	3. Administrator deletes the Solution number and indicates the system to save the changes.
	4. System indicates that the Solution number is missing.
Results:	The Solution's information did not get altered successfully.

Name:	**Administrator updates solution with incorrect data**
Scenario:	1. Administrator indicates that the Solution information need to be updated
	2. System makes the fields editable
	3. Administrator changes the number to "HEY" and indicates the system to save the changes.
	4. System indicates that the Solution number is incorrect.
Results:	The Solution's information did not get altered successfully.

Name:	**Administrator update solution with already existing data**
Scenario:	1. Administrator indicates that the Solution information need to be updated 2. System makes the fields editable 3. Administrator changes the Solution number to "2" and indicates the system to save the changes. 4. System indicates that the Solution number is already present in the system.
Results:	The Solution's information did not get altered successfully.

Name:	**Administrator successfully updates solution**
Scenario:	1. Administrator indicates that the Solution information need to be updated 2. System makes the fields editable 3. Administrator makes changes the name to "Solution A1" and indicates the system to save the changes. 4. System indicates that the Solution has been saved.
Results:	The Solution's information got altered successfully.

Delete Solution

Name:	**Administrator denies deletion of Solution**
Scenario:	1.Administrator indicates that the Solution "Solution A" should be deleted. 2. System warns the administrator by pointing out that this can't be undone. 3. Administrator denies the deletion. 4. System indicates that the Solution "Solution A" has not been deleted.

Name:	**Administrator denies deletion of Solution**
Results:	Solution was not successfully deleted.

Name:	**Actor approves deletion of Solution**
Scenario:	1.Administrator indicates that the Solution "Solution A" should be deleted. 2. System warns the administrator by pointing out that this can't be undone. 3. Administrator confirms the deletion. 4. System indicates that the Solution "Solution A" has been deleted, with all the information present.
Results:	Solution was successfully deleted.

Create Contact Person

Name:	**Administrator creates Contact Person with missing data**
Scenario:	1. Administrator indicates that a new contact person needs to be added 2. System displays general information form, containing: Last Name, First Name, Middle name(s), email(unique), phone number, Function/ Role, description (relational) 3. Administrator fills in "" as last name, "Hans" as first name, no middle name, hanspeter@gmail.com" as mail, "+31 4323432123" as phone number, "Lead Developer" as Function and no description. 4. System indicates that the contact person is missing the last name.
Results:	Adding the Contact Person was not successful.

Name:	**Administrator creates Contact Person with incorrect data**
Scenario:	1. Administrator indicates that a new contact person needs to be added

Name:	**Administrator creates Contact Person with incorrect data**
	2. System displays general information form, containing: Last Name, First Name, Middle name(s), email(unique), phone number, Function/ Role, description (relational)
	3. Administrator fills in "1234" as last name, "Hans" as first name, no middle name, hanspeter@gmail.com" as mail, "+31 4323432123" as phone number, "Lead Developer" as Function and no description.
	4. System indicates that the contact person's last name is incorrect.
Results:	Adding the Contact Person was not successful.
Name:	**Administrator creates Contact Person with already present data**
Scenario:	1. Administrator indicates that a new contact person needs to be added
	2. System displays general information form, containing: Last Name, First Name, Middle name(s), email(unique), phone number, Function/ Role, description (relational)
	3. Administrator fills in "Peter" as last name, "Hans" as first name, no middle name, karlheinz@gmail.com" as mail, "+31 4323432123" as phone number, "Lead Developer" as Function and no description.
	4. System indicates that the contact person'S mail is already present in the system.
Results:	Adding the Contact Person was not successful.
Name:	**Administrator creates Contact Person**
Scenario:	1. Administrator indicates that a new contact person needs to be added
	2. System displays general information form, containing: Last Name, First Name, Middle name(s), email(unique), phone number, Function/ Role, description (relational)
	3. Administrator fills in "Hans" as last name, "Hans" as first name, no middle name, hanspeter@gmail.com" as mail, "+31 4323432123" as phone number, "Lead Developer" as Function and no description.

Name:	**Administrator creates Contact Person**
	4. System indicates that the contact person was successfully added.
Results:	Adding the Contact Person was successful.

Read Contact Person

Name:	**Actor logs in with invalid combination of email and password**
Scenario:	1. System displays the customer's page.
	2. Administrator indicates he wants to view the information of "Hans Peter".
	3. System opens this specific person's page and displays the information to the administrator.
Results:	Administrator can see a "Hans Peter's" information

Update Person

Name:	**Administrator updates Contact Person with missing data**
Scenario:	1. Administrator indicates that the contact person's information need to be updated
	2. System makes the fields editable
	3. Administrator deletes the last name and indicates the system to save the changes.
	4. System indicates that the contact person is missing the last name.
Results:	The contact person's information did not get altered successfully.
Name:	**Administrator updates Contact Person with incorrect data**
Scenario:	1. Administrator indicates that the contact person's information need to be updated

Name:	**Administrator updates Contact Person with incorrect data**
	2. System makes the fields editable
	3. Administrator changes the first name to "112121212" and indicates the system to save the changes.
	4. System indicates that the contact person has an incorrect first name.
Results:	The contact person's information did not get altered successfully.
Name:	**Administrator updates Contact Person with already present data**
Scenario:	1. Administrator indicates that the contact person's information need to be updated
	2. System makes the fields editable
	3. Administrator updates the mail address to "karlheinz@gmail.com" and indicates the system to save the changes.
	4. System indicates that the mail address is already present in the system.
Results:	The contact person's information did not get altered successfully.
Name:	**Administrator updates Contact Person successfully**
Scenario:	1. Administrator indicates that the contact person's information need to be updated
	2. System makes the fields editable
	3. Administrator adds the middle name "Günter" and indicates the system to save the changes.
	4. System indicates that the contact person has been saved.
Results:	The contact person's information got altered successfully.

Delete Contact Person

Name:	**Administrator denies the deletion of the Contact Person**
Scenario:	1.Administrator indicates that the contact person "Hans Peter" should be deleted.
	2. System warns the administrator by pointing out that this can't be undone.
	3. Administrator denies the deletion.
	4. System indicates that the Contact Person has not been deleted.
Results:	Contact Person was not successfully deleted.

Name:	**Administrator approves the deletion of the Contact Person**
Scenario:	1.Administrator indicates that the contact person "Hans Peter" should be deleted.
	2. System warns the administrator by pointing out that this can't be undone.
	3. Administrator confirms the deletion.
	4. System indicates that the Contact Person has been deleted, with all the information present.
Results:	Contact Person was successfully deleted.